

```

1  <!DOCTYPE html>
2  <html lang="zh-CN">
3  <head>
4  <meta charset="UTF-8">
5  <meta http-equiv="Cache-Control" content="no-cache, no-store, must-revalidate">
6  <meta http-equiv="Pragma" content="no-cache">
7  <meta http-equiv="Expires" content="0">
8  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, use
r-scalable=no">
9  <title>■■■■■■■■</title>
10 <style>
11 @import url('https://fonts.googleapis.com/css2?family=Press+Start+2P&display=swap');
12 *{margin:0;padding:0;box-sizing:border-box;-webkit-tap-highlight-color:transparent}
13 html,body{width:100%;height:100%;overflow:hidden;background:#5c94fc;font-family:'Press Start
2P','Courier New','SimHei',monospace;touch-action:none;image-rendering:pixelated}
14 #game{display:block;position:absolute;top:0;left:0;image-rendering:pixelated}
15
16 #menu{position:absolute;inset:0;display:flex;flex-direction:column;align-items:center;justif
y-content:center;
17   background:linear-gradient(rgba(0,0,0,.35),rgba(0,0,0,.45)),url('assets/bg.png') center/co
ver no-repeat #3a2a1a;z-index:10;transition:opacity .4s}
18 #menu.hidden{opacity:0;pointer-events:none}
19 .menu-title{font-size:clamp(28px,6vw,48px);font-weight:700;color:#fff9c4;
20   -webkit-text-stroke:4px #f9a825;text-stroke:4px #f9a825;
21   paint-order:stroke fill;letter-spacing:0px;margin-bottom:8px;text-align:center}
22 .menu-sub{font-size:10px;color:#ffffff;-webkit-text-stroke:1px #333;text-stroke:1px #333;let
ter-spacing:2px;margin-bottom:48px;text-transform:uppercase;text-align:center}
23 .menu-info{font-size:15px;color:#ff2020;line-height:1.6;text-align:center;margin-bottom:40px
;max-width:420px;
24   background:rgba(30,0,0,.88);padding:24px 28px;border:3px solid #600000;border-radius:0;
25   box-shadow:0 0 30px rgba(200,0,0,.3),inset 0 0 30px rgba(0,0,0,.4);
26   animation:infoPulse 2s ease-in-out infinite;text-shadow:0 0 12px rgba(255,0,0,.6)}
27 @keyframes infoPulse{0%,100%{box-shadow:0 0 30px rgba(200,0,0,.3),inset 0 0 30px rgba(0,0,0
,.4)}50%{box-shadow:0 0 50px rgba(255,0,0,.5),inset 0 0 50px rgba(0,0,0,.5)}}
28 .menu-info b{color:#ff4040;text-shadow:0 0 8px rgba(255,0,0,.8)}
29 .menu-info .danger{color:#ff3030;font-weight:700;text-shadow:0 0 10px rgba(255,0,0,.9)}
30 .btn-start{padding:14px 48px;font-size:16px;font-weight:400;color:#ffffff;
31   background:#e82000;border:3px solid #600000;border-radius:0;cursor:pointer;
32   font-family:'Press Start 2P',monospace;box-shadow:4px 4px 0 rgba(0,0,0,.3);
33   transition:all .08s;letter-spacing:0px;text-shadow:2px 2px 0 rgba(0,0,0,.3)}
34 .btn-start:active{transform:translate(2px,2px);box-shadow:2px 2px 0 rgba(0,0,0,.3)}
35 .btn-start:hover{background:#ff4020}
36
37 #gameover{position:absolute;inset:0;display:flex;flex-direction:column;align-items:center;ju
stify-content:center;
38   background:rgba(0,0,0,.8);z-index:10;opacity:0;pointer-events:none;transition:opacity .5s}
39 #gameover.show{opacity:1;pointer-events:auto}
40 .go-title{font-size:clamp(28px,6vw,44px);font-weight:400;color:#f83800;
41   -webkit-text-stroke:2px #fff;text-stroke:2px #fff;paint-order:stroke fill;
42   letter-spacing:4px;text-shadow:3px 3px 0 #8b0000;margin-bottom:8px;text-align:center}
43 .go-sub{font-size:9px;color:#ccc;letter-spacing:3px;margin-bottom:36px;text-align:center}
44 .go-stats{display:flex;gap:32px;margin-bottom:40px;flex-wrap:wrap;justify-content:center}
45 .go-stat{text-align:center}
46 .go-stat-val{font-size:24px;font-weight:400;color:#ffd700;font-family:'Press Start 2P',monos
pace}
47 .go-stat-label{font-size:8px;color:#aaa;letter-spacing:1px;margin-top:6px;text-transform:upp
ercase}
48 .go-newrec{font-size:16px;font-weight:400;color:#ffd700;text-shadow:3px 3px 0 #8b6914;margin
-bottom:16px;animation:recglow 1s ease infinite alternate}
49 @keyframes recglow{from{text-shadow:2px 2px 0 #8b6914}to{text-shadow:4px 4px 0 #b8860b}}
50 .go-best{font-size:9px;color:#bbb;margin-bottom:24px}
51 .menu-best{font-size:9px;color:#ffffff;margin-bottom:16px;padding:8px 16px;
52   background:rgba(0,0,0,.3);border:2px solid #ffd700;box-shadow:0 0 0 rgba(0,0,0,0)}
53
54 /* HUD */
55 #hud{position:absolute;inset:0;pointer-events:none;z-index:5}
56 .hud-top{position:absolute;top:50px;left:0;right:0;padding:0 12px;display:flex;justify-conte
nt:space-between;align-items:flex-start}
57 .hud-left{display:flex;flex-direction:column;gap:5px}
58 .hud-level{font-size:12px;font-weight:400;color:#ffffff;-webkit-text-stroke:2px #333;text-st
roke:2px #333;paint-order:stroke fill}
59 .hud-weapons{display:flex;flex-wrap:wrap;gap:3px;margin-top:2px}
60 .hud-wp{font-size:8px;padding:2px 6px;border:2px solid #333;font-weight:400;white-space:nowr
ap;background:rgba(255,255,255,.8);color:#333}
61 .xp-bar{width:200px;height:10px;background:rgba(0,0,0,.3);border:2px solid #333;overflow:hid
den}
62 .xp-fill{height:100%;background:#f8b800;transition:width .15s}
63 .hud-xp{font-size:9px;color:#ffffff;-webkit-text-stroke:1px #333;text-stroke:1px #333;font-w

```

```

    eight:400}
64 .hud-right{display:flex;flex-direction:column;gap:4px;align-items:flex-end}
65 .hud-time{font-size:16px;font-weight:400;color:#ffffff;-webkit-text-stroke:2px #333;text-str
    oke:2px #333;paint-order:stroke fill}
66 .hud-kills{font-size:9px;color:#ffffff;-webkit-text-stroke:1px #333;text-stroke:1px #333}
67 .hud-crowns{font-size:11px;color:#ffd700;-webkit-text-stroke:1px #8b6914;text-stroke:1px #8b
    6914;font-weight:400}
68 .go-stat-crown .go-stat-val{color:#ffd700;text-shadow:2px 2px 0 #8b6914;animation:combopulse
    .5s ease infinite alternate}
69 .hud-combo{font-size:12px;font-weight:400;color:#ffd700;-webkit-text-stroke:2px #8b6914;text
    -stroke:2px #8b6914;animation:combopulse .3s ease}
70 @keyframes combopulse{0%{transform:scale(1.3)}100%{transform:scale(1)}}
71 .hud-dash{font-size:9px;color:#ffffff;margin-top:2px;-webkit-text-stroke:1px #333;text-strok
    e:1px #333}
72 .dash-key{display:inline-block;padding:1px 4px;border:2px solid #333;background:#f8b800;colo
    r:#333;font-weight:400;font-size:9px}
73 .hud-dash.cooling{opacity:.4}
74 .hud-dash.ready .dash-key{animation:dashready 1.5s ease infinite}
75 @keyframes dashready{0%,100%{box-shadow:0 0 0 rgba(248,184,0,0)}50%{box-shadow:0 0 8px rgba(
    248,184,0,.8)}}
76
77 /* ---- Mobile Dash Button ---- */
78 .dash-btn{
79     position:absolute;right:24px;bottom:24px;width:64px;height:64px;
80     background:#f8b800;border:4px solid #8b6914;
81     display:flex;flex-direction:column;align-items:center;justify-content:center;
82     z-index:6;user-select:none;-webkit-user-select:none;touch-action:none;
83     transition:transform .08s,box-shadow .2s;
84     color:#333;font-size:10px;font-weight:400;font-family:'Press Start 2P',monospace;
85     box-shadow:4px 4px 0 rgba(0,0,0,.3);
86 }
87 .dash-btn .dash-icon{font-size:20px;line-height:1;margin-bottom:1px}
88 .dash-btn .dash-cd{
89     position:absolute;inset:-4px;
90     background:conic-gradient(rgba(0,0,0,.6) var(--cd,0deg),transparent 0deg);
91     pointer-events:none;
92 }
93 .dash-btn.ready{box-shadow:4px 4px 0 rgba(0,0,0,.3),0 0 16px rgba(248,184,0,.6);animation:da
    shbtnpulse 1.5s ease infinite}
94 @keyframes dashbtnpulse{0%,100%{box-shadow:4px 4px 0 rgba(0,0,0,.3),0 0 8px rgba(248,184,0,.
    4)}50%{box-shadow:4px 4px 0 rgba(0,0,0,.3),0 0 20px rgba(248,184,0,.8)}}
95 .dash-btn.cooling{opacity:.5;box-shadow:2px 2px 0 rgba(0,0,0,.2)}
96 .dash-btn.cooling .dash-cd{display:block}
97 .dash-btn.ready .dash-cd{display:none}
98 .dash-btn.active{transform:translate(2px,2px);box-shadow:2px 2px 0 rgba(0,0,0,.2)}
99 .dash-btn.pressed{transform:translate(2px,2px);background:#ffcc00;box-shadow:2px 2px 0 rgba(
    0,0,0,.2)}
100
101 .btn-pause{position:absolute;top:10px;left:12px;width:36px;height:36px;
102     background:rgba(255,255,255,.7);border:3px solid #333;color:#333;
103     font-size:18px;display:flex;align-items:center;justify-content:center;cursor:pointer;
104     z-index:6;pointer-events:auto;transition:all .08s;font-family:'Press Start 2P',monospace}
105 .btn-pause:active{background:#fff;transform:scale(.9)}
106 .btn-music{position:absolute;top:10px;left:56px;width:36px;height:36px;
107     background:rgba(255,255,255,.7);border:3px solid #333;color:#333;
108     font-size:14px;display:flex;align-items:center;justify-content:center;cursor:pointer;
109     z-index:6;pointer-events:auto;transition:all .08s;font-family:'Press Start 2P',monospace}
110 .btn-music:active{background:#fff;transform:scale(.9)}
111
112 #pauseOverlay{position:absolute;inset:0;display:flex;flex-direction:column;align-items:cente
    r;justify-content:flex-start;
113     background:rgba(92,148,252,.95);z-index:3;opacity:0;pointer-events:none;transition:opacity
    .25s;overflow-y:auto;padding:48px 16px 32px}
114 #pauseOverlay.show{opacity:1;pointer-events:auto}
115 .pause-title{font-size:clamp(22px,5vw,36px);font-weight:400;color:#ffffff;
116     -webkit-text-stroke:2px #333;text-stroke:2px #333;paint-order:stroke fill;letter-spacing:3
    px;margin-bottom:6px}
117 .pause-hint{font-size:9px;color:#ffffff;letter-spacing:1px;margin-bottom:24px}
118 .pause-stats{display:flex;flex-wrap:wrap;justify-content:center;gap:12px;max-width:700px;wid
    th:100%}
119 .pause-weapon-card{background:rgba(255,255,255,.85);border:3px solid #8b4513;
120     padding:14px 16px;min-width:180px;max-width:220px;flex:1 1 180px;text-align:left;box-shado
    w:3px 3px 0 rgba(0,0,0,.2)}
121 .pw-header{display:flex;align-items:center;gap:8px;margin-bottom:10px}
122 .pw-icon{font-size:20px}
123 .pw-name{font-size:13px;font-weight:400;white-space:nowrap;overflow:hidden;text-overflow:ell
    ipsis;font-family:'Press Start 2P',monospace}
124 .pw-lv{font-size:9px;font-weight:400;padding:1px 6px;border:2px solid #333;background:rgba(2

```

```

55,255,255,.5);margin-left:auto;flex-shrink:0}
125 .pw-row{display:flex;justify-content:space-between;font-size:9px;color:#555;margin-bottom:4p
x}
126 .pw-row span:last-child{color:#222;font-weight:400}
127 .pw-type-badge{display:inline-block;font-size:8px;padding:1px 6px;border:2px solid #333;marg
in-bottom:8px;font-weight:400;letter-spacing:1px;color:#333}
128 .pw-type-projectile{background:#ffcc44}
129 .pw-type-orbital{background:#ff8844}
130 .pw-type-pulse{background:#cc88ff}
131 .pw-abilities{font-size:8px;color:#555;margin-top:6px;display:flex;flex-wrap:wrap;gap:3px}
132 .pw-abi{display:inline-block;padding:0 4px;border:1px solid #aaa;color:#666}
133
134 .pause-bottom-btns{display:flex;justify-content:center;gap:16px;margin-top:28px;width:100%;m
ax-width:700px;flex-wrap:wrap}
135 .pause-btn{border:3px solid #6b3300;background:#f8b800;color:#6b3300;
font-family:'Press Start 2P','Courier New','SimHei',monospace;font-size:10px;
padding:10px 18px;cursor:pointer;box-shadow:3px 3px 0 #6b3300;
letter-spacing:1px;text-transform:uppercase}
139 .pause-btn:active{background:#fff;transform:translate(2px,2px);box-shadow:1px 1px 0 #6b3300}
140
141 #minimap{position:absolute;bottom:12px;right:12px;width:120px;height:120px;
border:3px solid #8b4513;background:rgba(92,148,252,.7);z-index:5;image-rendering:pixelate
d;box-shadow:3px 3px 0 rgba(0,0,0,.3)}
142
143
144 /* Upgrade Modal */
145 #upgradeModal{position:absolute;inset:0;display:flex;flex-direction:column;align-items:cente
r;justify-content:center;
background:rgba(0,0,0,.7);z-index:20;opacity:0;pointer-events:none;transition:opacity .3s}
147 #upgradeModal.show{opacity:1;pointer-events:auto}
148 .upgrade-title{font-size:20px;font-weight:400;color:#ffd700;
-webkit-text-stroke:2px #8b6914;text-stroke:2px #8b6914;paint-order:stroke fill;
letter-spacing:2px;margin-bottom:24px;text-shadow:3px 3px 0 #564208}
151 .upgrade-choices{display:flex;gap:16px;flex-wrap:wrap;justify-content:center;padding:0 16px}
152 .upgrade-card{width:280px;max-width:45vw;padding:20px 16px;cursor:pointer;
border:3px solid #333;transition:all .08s;position:relative;overflow:hidden;box-shadow:4px
4px 0 rgba(0,0,0,.3)}
154 .upgrade-card:active{transform:translate(2px,2px);box-shadow:2px 2px 0 rgba(0,0,0,.3)}
155 .upgrade-card:hover{border-color:#f8b800}
156 .upgrade-card .card-rarity{font-size:9px;letter-spacing:1px;text-transform:uppercase;margin-
bottom:4px}
157 .upgrade-card .card-icon{font-size:36px;margin-bottom:8px}
158 .upgrade-card .card-name{font-size:16px;font-weight:400;margin-bottom:6px;font-family:'Press
Start 2P',monospace;color:#222}
159 .upgrade-card .card-desc{font-size:10px;color:#555;line-height:1.6}
160 .upgrade-card .card-stat{display:inline-block;padding:2px 6px;margin:2px;font-size:9px;font-
weight:400;color:#fff;border:2px solid #333}
161 .upgrade-card.common{background:#e8e0d0}
162 .upgrade-card.common .card-rarity{color:#8b7355}
163 .upgrade-card.common .card-stat{background:#8b7355}
164 .upgrade-card.rare{background:#d4e8f0}
165 .upgrade-card.rare .card-rarity{color:#3060a0}
166 .upgrade-card.rare .card-stat{background:#3060a0}
167 .upgrade-card.epic{background:#e4d0f4}
168 .upgrade-card.epic .card-rarity{color:#7030a0}
169 .upgrade-card.epic .card-stat{background:#7030a0}
170 .upgrade-card.legendary{background:#ffe8b0}
171 .upgrade-card.legendary .card-rarity{color:#b8860b}
172 .upgrade-card.legendary .card-stat{background:#b8860b}
173
174 .toast{position:absolute;top:35%;left:50%;transform:translate(-50%,-50%) scale(.8);
font-size:18px;font-weight:400;color:#ffd700;
-webkit-text-stroke:2px #8b6914;text-stroke:2px #8b6914;paint-order:stroke fill;
opacity:0;pointer-events:none;z-index:15;transition:all .3s;text-align:center;white-space:
nowrap;
font-family:'Press Start 2P',monospace}
179 .toast.show{opacity:1;transform:translate(-50%,-50%) scale(1)}
180 .toast .toast-sub{font-size:10px;color:#ffffff;-webkit-text-stroke:1px #333;text-stroke:1px
#333;font-weight:400;margin-top:4px;letter-spacing:1px}
181 .joy-hint{position:absolute;bottom:20px;left:20px;font-size:8px;color:#ffffff;-webkit-text-s
troke:1px #333;text-stroke:1px #333;z-index:5;pointer-events:none}
182 </style>
183 </head>
184 <body>
185 <canvas id="game"></canvas>
186
187 <div id="hud">
188   <div class="hud-top">
189     <div class="hud-left">

```

[illegible]

```
263 const miniCtx = miniCanvas.getContext('2d');
264 let W, H, DPR;
265
266 function resize() {
267   DPR = window.devicePixelRatio || 1;
268   W = window.innerWidth;
269   H = window.innerHeight;
270   canvas.width = W * DPR;
271   canvas.height = H * DPR;
272   canvas.style.width = W + 'px';
273   canvas.style.height = H + 'px';
274   ctx.setTransform(DPR, 0, 0, DPR, 0, 0);
275 }
276 window.addEventListener('resize', resize);
277 resize();
278
279 // ---- Map Textures (Mario pixel style) ----
280 let skyTex = null, groundTex = null;
281 function generateMapTextures() {
282   const sc = document.createElement('canvas'); sc.width = 128; sc.height = 128;
283   const st = sc.getContext('2d');
284   st.fillStyle = '#5c94fc'; st.fillRect(0, 0, 128, 128);
285   for (let i = 0; i < 6; i++) {
286     st.fillStyle = `rgba(255,255,255,${0.04 + Math.random() * 0.06})`;
287     st.beginPath(); st.arc(Math.random() * 128, Math.random() * 128, 16 + Math.random() * 20
, 0, Math.PI * 2); st.fill();
288   }
289   skyTex = sc;
290
291   const gc = document.createElement('canvas'); gc.width = 128; gc.height = 64;
292   const gt = gc.getContext('2d');
293   gt.fillStyle = '#a04c10'; gt.fillRect(0, 0, 128, 64);
294   gt.fillStyle = '#8b4513'; gt.fillRect(0, 0, 128, 8);
295   for (let x = 0; x < 128; x += 4) {
296     const gh = 2 + Math.sin(x * 0.3) * 2 + Math.cos(x * 0.7) * 1;
297     gt.fillStyle = `rgb(${30 + Math.sin(x * 0.5) * 10},${120 + Math.sin(x * 0.4) * 15},${20
+ Math.sin(x * 0.6) * 10})`;
298     gt.fillRect(x, 8, 4, gh);
299   }
300   gt.fillStyle = '#6b8c20'; gt.fillRect(0, 8, 128, 6);
301   for (let y = 14; y < 64; y += 4) {
302     for (let x = 0; x < 128; x += 4) {
303       gt.fillStyle = `rgba(0,0,0,${0.02 + Math.random() * 0.04})`;
304       gt.fillRect(x, y, 4, 4);
305     }
306   }
307   groundTex = gc;
308 }
309 generateMapTextures();
310
311 // ---- Background Image ----
312 const bgImg = new Image();
313 bgImg.onload = () => { bgImg.ready = true; };
314 bgImg.onerror = () => { console.warn('Failed to load background image'); };
315 bgImg.src = 'assets/bg.png';
316
317 // ---- Weapon Sprite Sheets ----
318 const SPRITE_SHEETS = {
319   projectiles: { img: null, cellW: 112, cellH: 112, cols: 7, rows: 2, path: 'assets/projecti
les.png' },
320   orbitals: { img: null, cellW: 112, cellH: 112, cols: 7, rows: 1, path: 'assets/orbitals
.png' },
321   fields: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 2, path: 'assets/fields.p
ng' },
322   player: { img: null, cellW: 64, cellH: 64, cols: 4, rows: 1, path: 'assets/player.p
ng' },
323   meteor: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/meteor.p
ng' },
324   magnet: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/magnet.p
ng' },
325   starfield: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/starfiel
d.png' },
326   static: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/static.p
ng' },
327   force: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/force.pn
g' },
328   cosmic: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/cosmic.p
ng' },
```

```

329   nova:      { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/nova.png
330   },
331   blackhole: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/blackhol
332   e.png' },
333   starburst: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/starburs
334   t.png' },
335   shell:     { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/shell.pn
336   g' },
337   iron:      { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/iron.png
338   ' },
339   lava:      { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/lava.png
340   ' },
341   boomerang: { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/boomeran
342   g.png' },
343   ring:      { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/ring.png
344   ' },
345   star:      { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/star.png
346   ' },
347   flail:     { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/flail.pn
348   g' },
349   solar:     { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/solar.pn
350   g' },
351   shield:    { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/shield.p
352   ng' },
353   fireball:  { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/fireball
354   .png' },
355   pellet:    { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/pellet.p
356   ng' },
357   cannon:    { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/cannon.p
358   ng' },
359   laser:     { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/laser.pn
360   g' },
361   spark:     { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/spark.pn
362   g' },
363   ember:     { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/ember.pn
364   g' },
365   dragon:    { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/dragon.p
366   ng' },
367   hellfire:  { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/hellfire
368   .png' },
369   phoenix:   { img: null, cellW: 112, cellH: 112, cols: 4, rows: 1, path: 'assets/phoenix.
370   png' },
371   };
372   let spritesLoaded = 0;
373   let spritesAllLoaded = false;
374
375   const WEAPON_SPRITE_MAP = {
376     // Projectiles (sheet projectiles)
377     'fire':    { sheet: 'fireball', col: 0, row: 0 },
378     'spark':   { sheet: 'spark', col: 0, row: 0 },
379     'ember':   { sheet: 'ember', col: 0, row: 0 },
380     'laser':   { sheet: 'laser', col: 0, row: 0 },
381     'cannon':  { sheet: 'cannon', col: 0, row: 0 },
382     'pellet':  { sheet: 'pellet', col: 0, row: 0 },
383     'lava':    { sheet: 'lava', col: 0, row: 0 },
384     'storm':   { sheet: 'ember', col: 0, row: 0 },
385     'phoenix': { sheet: 'phoenix', col: 0, row: 0 },
386     'dragon':  { sheet: 'dragon', col: 0, row: 0 },
387     'solar':   { sheet: 'solar', col: 0, row: 0 },
388     'hellfire': { sheet: 'hellfire', col: 0, row: 0 },
389     'meteor':  { sheet: 'ember', col: 0, row: 0 },
390     // Orbitals (sheet orbitals)
391     'shell':   { sheet: 'shell', col: 0, row: 0 },
392     'iron':    { sheet: 'iron', col: 0, row: 0 },
393     'boomerang': { sheet: 'boomerang', col: 0, row: 0 },
394     'flail':   { sheet: 'flail', col: 0, row: 0 },
395     'star':    { sheet: 'star', col: 0, row: 0 },
396     'ring':    { sheet: 'ring', col: 0, row: 0 },
397     'shield':  { sheet: 'shield', col: 0, row: 0 },
398     // Fields (sheet fields)
399     'magnet':  { sheet: 'magnet', col: 0, row: 0 },
400     'starfield': { sheet: 'starfield', col: 0, row: 0 },
401     'static':  { sheet: 'static', col: 0, row: 0 },
402     'force':   { sheet: 'force', col: 0, row: 0 },
403     'cosmic':  { sheet: 'cosmic', col: 0, row: 0 },
404     'nova':    { sheet: 'nova', col: 0, row: 0 },
405     'blackhole': { sheet: 'blackhole', col: 0, row: 0 },
406     'starburst': { sheet: 'starburst', col: 0, row: 0 },

```

```

386 };
387
388 function loadSpriteSheets() {
389     let remaining = Object.keys(SPRITE_SHEETS).length;
390     for (const key of Object.keys(SPRITE_SHEETS)) {
391         const sheet = SPRITE_SHEETS[key];
392         const img = new Image();
393         img.onload = () => {
394             sheet.img = img;
395             spritesLoaded++;
396             if (--remaining === 0) spritesAllLoaded = true;
397         };
398         img.onerror = () => {
399             console.warn('Failed to load sprite sheet:', sheet.path);
400             if (--remaining === 0) spritesAllLoaded = true;
401         };
402         img.src = sheet.path;
403     }
404 }
405 loadSpriteSheets();
406
407 function drawSpriteFromSheet(sheetKey, col, row, x, y, w, h, rotate) {
408     const sheet = SPRITE_SHEETS[sheetKey];
409     if (!sheet || !sheet.img) return false;
410     const cw = sheet.cellW, ch = sheet.cellH;
411     const sx = col * cw, sy = row * ch;
412     ctx.save();
413     ctx.imageSmoothingEnabled = false;
414     ctx.translate(x, y);
415     if (rotate !== undefined) ctx.rotate(rotate);
416     ctx.drawImage(sheet.img, sx, sy, cw, ch, -w / 2, -h / 2, w, h);
417     ctx.restore();
418     return true;
419 }
420
421 function drawWeaponSprite(vStyle, x, y, w, h, rotate) {
422     const map = WEAPON_SPRITE_MAP[vStyle];
423     if (!map) return false;
424     const sheet = SPRITE_SHEETS[map.sheet];
425     const totalFrames = sheet ? sheet.cols : 1;
426     const frame = totalFrames > 1 ? (Math.floor(game.frame / 6) % totalFrames) : 0;
427     return drawSpriteFromSheet(map.sheet, frame, map.row, x, y, w, h, rotate);
428 }
429
430 // ---- Config ----
431 const CFG = {
432     MAP_W: 4000, MAP_H: 4000, PLAYER_SPEED: 3.6, PLAYER_RADIUS: 13,
433     BASE_XP: 20, MAX_MONSTERS: 200, MAX_PARTICLES: 400, PROJECTILE_RANGE: 500,
434     MAX_WEAPONS: 6,
435 };
436 function xpNeeded(level) { return CFG.BASE_XP + (level - 1) * (level + 3); }
437
438 // =====
439 // DIFFICULTY CURVE – Three-phase design
440 // Phase 1 (0-80s): high pressure, fast spawns, HP grows
441 // Phase 2 (80-200s): breather, slow HP growth, weapons catch up
442 // Phase 3 (200s+): escalating, faster & tankier monsters
443 // =====
444 function diffSpawnInterval(t) {
445     if (t < 20) return Math.max(35, Math.floor(40 - t * 0.25)); // 40→35 (halved rat
446     e for first 20s)
447     if (t < 80) return Math.max(14, Math.floor(35 - (t-20) * 0.35)); // 35→14
448     if (t < 200) return Math.max(10, Math.floor(14 - (t-80) * 0.033)); // 14→10
449     return Math.max(4, Math.floor(10 - (t-200) * 0.045)); // 10→4
450 }
451 function diffSpawnBatch(t) {
452     if (t < 80) return 1 + Math.floor(t / 44); // 1→2
453     if (t < 200) return Math.min(4, 2 + Math.floor((t-80) / 48)); // 2→4
454     return Math.min(6, 4 + Math.floor((t-200) / 50)); // 4→6
455 }
456 function diffHpMult(t) {
457     // 3 minutes: exponential apocalypse – ×10 every minute
458     if (t >= 180) return Math.pow(10, 1 + Math.floor((t - 180) / 60));
459     // Before 3 min: smooth growth
460     if (t < 80) return 1 + t * 0.012; // 1.0→1.96x
461     if (t < 180) return 1.96 + (t-80) * 0.0045; // 1.96→2.41x at 180s
462     return 1; // unreachable
463 }

```

```

463 function diffSpeedMult(t) {
464   if (t < 180) return 1.0;
465   // After 3 min: no multiplicative speed scaling (bonus handled separately)
466   return 1.0;
467 }
468 function diffSpeedBonus(t) {
469   if (t < 180) return 0;
470   return 1; // +1 flat speed to all monsters after 3 min
471 }
472 function diffEliteChance(t) {
473   if (t < 30) return 0;
474   if (t < 200) return Math.min(0.07, 0.02 + (t-30) * 0.00033); // 0→7%
475   return Math.min(0.20, 0.07 + (t-200) * 0.0008); // 7→20%
476 }
477 function diffBossInterval(t) {
478   if (t < 200) return 90;
479   return Math.max(40, 90 - (t-200) * 0.35); // 90→40s
480 }
481
482 // =====
483 // MAP OBSTACLES – Brick blocks & pipes for terrain strategy
484 // =====
485 const OBSTACLE_COUNT = 45;
486 function generateObstacles() {
487   const obs = [];
488   const centerX = CFG.MAP_W / 2, centerY = CFG.MAP_H / 2;
489   const safeR = 200; // safe zone around spawn
490
491   for (let i = 0; i < OBSTACLE_COUNT; i++) {
492     let x, y, w, h, type;
493     const isPipe = Math.random() < 0.28;
494     if (isPipe) {
495       w = 30 + Math.floor(Math.random() * 20); h = 40 + Math.floor(Math.random() * 30);
496       type = 'pipe';
497     } else {
498       const cluster = Math.random() < 0.5 ? 2 : 1;
499       w = 36 + Math.floor(Math.random() * 28);
500       h = cluster > 1 ? (18 + Math.floor(Math.random() * 20)) : w;
501       type = 'brick';
502     }
503     // Place randomly, avoid center safe zone
504     let attempts = 0;
505     do {
506       x = rand(40, CFG.MAP_W - w - 40);
507       y = rand(40, CFG.MAP_H - h - 40);
508       attempts++;
509     } while (attempts < 50 && Math.hypot(x + w/2 - centerX, y + h/2 - centerY) < safeR + 50)
510   };
511   // Check overlap with existing obstacles (min 30px gap)
512   const tooClose = obs.some(o =>
513     Math.abs(x + w/2 - o.x - o.w/2) < (w + o.w)/2 + 30 &&
514     Math.abs(y + h/2 - o.y - o.h/2) < (h + o.h)/2 + 30
515   );
516   if (!tooClose) obs.push({ x, y, w, h, type });
517   else i--;
518 }
519 return obs;
520 }
521
522 function rectCircleCollision(rx, ry, rw, rh, cx, cy, cr) {
523   const nearestX = Math.max(rx, Math.min(cx, rx + rw));
524   const nearestY = Math.max(ry, Math.min(cy, ry + rh));
525   const dx = cx - nearestX, dy = cy - nearestY;
526   return dx * dx + dy * dy < cr * cr;
527 }
528
529 function pushOutOfObstacles(entity, radius) {
530   for (const o of game.obstacles) {
531     if (rectCircleCollision(o.x, o.y, o.w, o.h, entity.x, entity.y, radius)) {
532       // Push entity to nearest edge
533       const cx = entity.x, cy = entity.y;
534       const toLeft = Math.abs(cx - o.x), toRight = Math.abs(cx - (o.x + o.w));
535       const toTop = Math.abs(cy - o.y), toBottom = Math.abs(cy - (o.y + o.h));
536       const minDist = Math.min(toLeft, toRight, toTop, toBottom);
537       if (minDist === toLeft) entity.x = o.x - radius;
538       else if (minDist === toRight) entity.x = o.x + o.w + radius;
539       else if (minDist === toTop) entity.y = o.y - radius;

```



```

540     else entity.y = o.y + o.h + radius;
541     entity.x = clamp(entity.x, radius, CFG.MAP_W - radius);
542     entity.y = clamp(entity.y, radius, CFG.MAP_H - radius);
543 }
544 }
545 }
546
547 // =====
548 // WEAPON SYSTEM - Roguelike style
549 // =====
550
551 let weaponUid = 0;
552
553 // Weapon type base templates
554 const WEAPON_BASES = {
555   projectile: {
556     name: '■■', icon: '■', type: 'projectile',
557     damage: 3, cooldown: 550, speed: 10, count: 1, spread: 0.05,
558     piercing: false, bulletRadius: 5,
559     colors: ['#ff8833', '#ff6622', '#ffaa00', '#ffcc00', '#ff4400', '#ff7733', '#ff9944', '#ffcc44',
560             '#ff5500', '#e83800', '#ffdd44', '#ff6600', '#ff8811'],
561   },
562   orbital: {
563     name: '■■', icon: '■', type: 'orbital',
564     damage: 5, cooldown: 300, count: 1, orbitSpeed: 0.025, orbitRadius: 90,
565     size: 10,
566     colors: ['#ffaa44', '#ff8833', '#44ff66', '#ffcc00', '#ff4488', '#88ff44', '#44aaff', '#ff44aa'
567   ],
568   field: {
569     name: '■■', icon: '■', type: 'field',
570     damage: 4, cooldown: 500, fieldRadius: 120, size: 4,
571     colors: ['#ffd700', '#ffaa00', '#ffcc00', '#f8b800', '#ffdd44', '#e8a800', '#ffc830', '#ffb800'
572   ],
573 },
574 // Weapon name traits – each weapon name has unique base stat modifiers
575 // Modifiers are ADDITIVE for most stats, MULTIPLIER (*Mul) for orbital/field
576 const WEAPON_TRAITS = {
577   projectile: {
578     // Base: dmg:2, cd:550, spd:10, cnt:1, sprd:0.05, prc:false, bRad:5
579     '■■■■': {dmg:0, cd:0, spd:0, cnt:0, sprd:0, prc:false, bRad:0, vStyle:'fire'},
580     '■■■■': {dmg:0, cd:-80, spd:3, cnt:0, sprd:-0.02, prc:false, bRad:-1, vStyle:'spark'},
581     '■■■■': {dmg:0, cd:0, spd:-1, cnt:2, sprd:0.08, prc:false, bRad:0, vStyle:'ember'},
582     '■■■■': {dmg:1, cd:50, spd:2, cnt:0, sprd:-0.03, prc:true, bRad:-1, vStyle:'laser'},
583     '■■■■': {dmg:2, cd:200, spd:0, cnt:0, sprd:0, prc:false, bRad:2, vStyle:'cannon'}
584   },
585   '■■■■': {dmg:0, cd:0, spd:-1, cnt:1, sprd:0.03, prc:false, bRad:0, vStyle:'pellet'},
586   '■■■■': {dmg:1, cd:100, spd:-2, cnt:0, sprd:0, prc:false, bRad:3, vStyle:'lava'},
587   '■■■■': {dmg:1, cd:100, spd:2, cnt:0, sprd:0, prc:false, bRad:1, vStyle:'meteor'}
588 },
589 '■■■■': {dmg:0, cd:0, spd:1, cnt:3, sprd:0.10, prc:false, bRad:-1, vStyle:'storm'},
590 '■■■■': {dmg:1, cd:-50, spd:1, cnt:0, sprd:0, prc:false, bRad:1, vStyle:'phoenix'}
591 },
592 '■■■■': {dmg:2, cd:150, spd:0, cnt:1, sprd:0.02, prc:false, bRad:0, vStyle:'dragon'}
593 },
594 '■■■■': {dmg:3, cd:250, spd:4, cnt:0, sprd:-0.04, prc:true, bRad:-1, vStyle:'solar'},
595 '■■■■': {dmg:0, cd:-50, spd:0, cnt:4, sprd:0.12, prc:false, bRad:-2, vStyle:'hellfire'}
596 },
597 },
598 orbital: {
599   // Base: dmg:4, cd:300, cnt:1, spd:0.025, rad:90, size:10. spdMul/radMul = MULTIPLIERS.
600   '■■': {dmg:2, spdMul:0.7, radMul:0.7, cnt:0, size:3, vStyle:'shell'},
601   '■■■■': {dmg:1, spdMul:1.0, radMul:1.0, cnt:0, size:0, vStyle:'iron'},
602   '■■■■': {dmg:0, spdMul:1.5, radMul:1.1, cnt:0, size:-2, vStyle:'boomerang'},
603   '■■': {dmg:0, spdMul:0.8, radMul:1.4, cnt:0, size:2, vStyle:'flail'},
604   '■■■■': {dmg:0, spdMul:1.0, radMul:1.0, cnt:1, size:-1, vStyle:'star'},
605   '■■■■': {dmg:2, spdMul:1.3, radMul:0.8, cnt:0, size:-1, vStyle:'ring'},
606   '■■■■': {dmg:1, spdMul:0.7, radMul:1.3, cnt:0, size:1, vStyle:'shield'},
607 },
608 field: {
609   // Base: dmg:3, cd:500, rad:120. cdMul/radMul = MULTIPLIERS.
610   '■■■■': {dmg:0, cdMul:1.0, radMul:1.0, vStyle:'magnet'},
611   '■■■■': {dmg:0, cdMul:0.7, radMul:0.8, vStyle:'starfield'},
612   '■■■■': {dmg:0, cdMul:1.0, radMul:1.4, vStyle:'static'},
613   '■■■■': {dmg:2, cdMul:1.2, radMul:0.7, vStyle:'force'},
614   '■■■■': {dmg:0, cdMul:0.8, radMul:1.0, vStyle:'cosmic'},

```

```

610     '■■■■': {dmg:0, cdMul:1.3, radMul:1.6, vStyle:'nova'},
611     '■■■■': {dmg:3, cdMul:1.4, radMul:1.1, vStyle:'blackhole'},
612     '■■':   {dmg:1, cdMul:0.8, radMul:1.1, vStyle:'starburst'},
613 },
614 };
615
616 // Themed upgrade names per weapon name + upgrade archetype
617 const WEAPON_UPGRADE_THEMES = {
618   projectile: {
619     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'■■■■', storm:'■■■■'},
620     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'■■■■', storm:'■■■■'},
621     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■', scatter:'--', storm:'■■■■'},
622     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'--', scatter:'■■■■', storm:'■■■■'},
623     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'■■■', storm:'■■■■'},
624     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■', scatter:'■■■■', storm:'■■■■'},
625     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'■■■■', storm:'■■■■'},
626     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'■■■■', storm:'■■■■'},
627     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■', scatter:'--', storm:'■■■■'},
628     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'■■■■', storm:'■■■■'},
629     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'■■■■', storm:'■■■■'},
630     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'--', scatter:'■■■■', storm:'■■■■'},
631     '■■■■': {damage:'■■■■', multi:'■■■■', speed:'■■■■', pierce:'■■■■', scatter:'--', storm:'■■■■'},
632   },
633   orbital: {
634     '■■': {damage:'■■■■', twin:'■■■■', range:'■■■■', speed:'■■■■', gravity:'■■■■', cluster:'■■■■'},
635     '■■■■': {damage:'■■■■', twin:'■■■', range:'■■■■', speed:'■■■■', gravity:'■■■■', cluster:'■■■'},
636     '■■■■': {damage:'■■■■', twin:'■■■■', range:'■■■■', speed:'■■■■', gravity:'■■■■', cluster:'■■■■'},
637     '■■': {damage:'■■■■', twin:'■■■', range:'■■■■', speed:'■■■■', gravity:'■■■■', cluster:'■■■■'},
638     '■■■■': {damage:'■■■■', twin:'■■■■', range:'■■■■', speed:'■■■■', gravity:'■■■■', cluster:'■■■■'},
639     '■■■■': {damage:'■■■■', twin:'■■■', range:'■■■■', speed:'■■■■', gravity:'■■■■', cluster:'■■■■'},
640     '■■■■': {damage:'■■■■', twin:'■■■■', range:'■■■■', speed:'■■■■', gravity:'■■■■', cluster:'■■■■'},
641   },
642   field: {
643     '■■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■■', storm:'■■', singularity:'■■■■'},
644     '■■■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■■', storm:'■■■■', singularity:'■■■■'},
645     '■■■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■■', storm:'■■■■', singularity:'■■■■'},
646     '■■■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■■', storm:'■■■■', singularity:'■■■■'},
647     '■■■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■■', storm:'■■■■', singularity:'■■■■'},
648     '■■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■■', storm:'■■■■', singularity:'■■■■'},
649     '■■■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■', storm:'■■■■', singularity:'■■■■'},
650     '■■': {damage:'■■■■', power:'■■■■', expand:'■■■■', pulse:'■■■■', storm:'■■■■', singularity:'■■■■'},
651   },
652 };
653
654 function createWeapon(type) {
655   const base = WEAPON_BASES[type];
656   const traitMap = WEAPON_TRAITS[type];
657   const nameKeys = Object.keys(traitMap);
658   const name = nameKeys[Math.floor(Math.random() * nameKeys.length)];
659   const t = traitMap[name];

```

```

660     const color = base.colors[Math.floor(Math.random() * base.colors.length)];
661
662     let damage = base.damage + (t.dmg || 0);
663     if (damage < 1) damage = 1; // Minimum 1 damage
664
665     let cooldown, speed, count, spread, piercing, bulletRadius, size;
666     let orbitSpeed, orbitRadius, fieldRadius;
667
668     if (type === 'projectile') {
669         cooldown = Math.max(80, base.cooldown + (t.cd || 0));
670         speed = Math.max(4, (base.speed || 10) + (t.spd || 0));
671         count = 1 + (t.cnt || 0);
672         spread = Math.max(0, (base.spread || 0.05) + (t.sprd || 0));
673         piercing = t.prc || false;
674         bulletRadius = Math.max(3, Math.min(20, (base.bulletRadius || 5) + (t.bRad || 0) + Math.
        floor(damage * 1.2)));
675         orbitSpeed = base.orbitSpeed || 0.03;
676         orbitRadius = base.orbitRadius || 55;
677         size = (base.size || 8) + Math.floor(damage * 1.5);
678         fieldRadius = base.fieldRadius || 120;
679     } else if (type === 'orbital') {
680         orbitSpeed = Math.max(0.005, (base.orbitSpeed || 0.025) * (t.spdMul || 1));
681         orbitRadius = Math.max(20, Math.floor((base.orbitRadius || 90) * (t.radMul || 1)));
682         size = Math.max(4, Math.min(28, (base.size || 10) + (t.size || 0) + Math.floor(damage *
        1.5)));
683         count = 1 + (t.cnt || 0);
684         cooldown = base.cooldown || 300;
685         speed = base.speed || 10;
686         spread = base.spread || 0;
687         piercing = base.piercing || false;
688         bulletRadius = (base.bulletRadius || 5) + Math.floor(damage * 1.2);
689         fieldRadius = base.fieldRadius || 120;
690     } else if (type === 'field') {
691         fieldRadius = Math.max(30, Math.floor((base.fieldRadius || 120) * (t.radMul || 1)));
692         cooldown = Math.max(80, Math.floor((base.cooldown || 500) * (t.cdMul || 1)));
693         count = base.count || 1;
694         speed = base.speed || 10;
695         spread = base.spread || 0;
696         piercing = base.piercing || false;
697         bulletRadius = (base.bulletRadius || 5) + Math.floor(damage * 1.2);
698         orbitSpeed = base.orbitSpeed || 0.03;
699         orbitRadius = base.orbitRadius || 55;
700         size = (base.size || 8) + Math.floor(damage * 1.5);
701     }
702
703     const w = {
704         id: ++weaponUid,
705         type: base.type,
706         name: name,
707         vStyle: t.vStyle || 'fire', // visual style for unique appearance
708         icon: base.icon,
709         level: 1,
710         damage, cooldown, timer: 0,
711         color: color, baseColor: color,
712         count, speed, spread, piercing, bulletRadius,
713         orbitSpeed, orbitRadius, size,
714         angle: Math.random() * Math.PI * 2,
715         fieldRadius,
716         vfx: {
717             muzzleFlash: false, energyRing: false, piercingGlow: false,
718             doubleRing: false, sparkTrail: false, ripple: false,
719             heatWave: false, scatterBurst: false,
720         },
721     };
722     return w;
723 }
724
725 // ---- Upgrade Choice Generator ----
726 function generateUpgradeChoices() {
727     const p = game.player;
728     const choices = [];
729
730     const ALL_TYPES = ['projectile', 'orbital', 'field'];
731     const ownedTypes = p.weapons.map(w => w.type);
732     const newTypes = ALL_TYPES.filter(t => !ownedTypes.includes(t));
733     const canAddNew = p.weapons.length < CFG.MAX_WEAPONS && newTypes.length > 0;
734
735     function fingerprint(c) {

```

```
736     if (c.kind === 'new') return 'new_' + c.type;
737     return 'upgrade_' + p.weapons.indexOf(c.targetWeapon) + '_' + c.name;
738 }
739
740 function isDup(candidate) {
741     const fp = fingerprint(candidate);
742     return choices.some(existing => fingerprint(existing) === fp);
743 }
744
745 function makeOneChoice(retries) {
746     if (retries === undefined) retries = 10;
747     if (retries <= 0) return null;
748
749     if (canAddNew && Math.random() < 0.45) {
750         const type = newTypes[Math.floor(Math.random() * newTypes.length)];
751         const wp = createWeapon(type);
752         return {
753             kind: 'new', name: wp.name, icon: wp.icon, type: type,
754             typeLabel: WEAPON_BASES[type].name, desc: getTypeDescription(type),
755             rarity: 'rare', weapon: wp,
756         };
757     } else if (p.weapons.length > 0) {
758         const wp = p.weapons[Math.floor(Math.random() * p.weapons.length)];
759         const upgrade = generateUpgrade(wp);
760         if (upgrade) return upgrade;
761         if (canAddNew) {
762             const type = newTypes[Math.floor(Math.random() * newTypes.length)];
763             const nw = createWeapon(type);
764             return { kind: 'new', name: nw.name, icon: nw.icon, type: type,
765                 typeLabel: WEAPON_BASES[type].name, desc: getTypeDescription(type),
766                 rarity: 'rare', weapon: nw };
767         }
768     }
769     return null;
770 }
771
772 for (let i = 0; i < 2; i++) {
773     let attempt = 0;
774     let candidate = null;
775     do {
776         candidate = makeOneChoice(10 - attempt);
777         attempt++;
778     } while (candidate && isDup(candidate) && attempt < 10);
779     if (candidate) choices.push(candidate);
780 }
781
782 // Fallback to fill up to 2 choices
783 while (choices.length < 2 && canAddNew) {
784     const type = newTypes[Math.floor(Math.random() * newTypes.length)];
785     const nw = createWeapon(type);
786     const cand = { kind: 'new', name: nw.name, icon: nw.icon, type: type,
787         typeLabel: WEAPON_BASES[type].name, desc: getTypeDescription(type),
788         rarity: 'common', weapon: nw };
789     if (!isDup(cand)) choices.push(cand); else break;
790 }
791 while (choices.length < 2 && p.weapons.length > 0) {
792     const wp = p.weapons[Math.floor(Math.random() * p.weapons.length)];
793     const upgrade = generateUpgrade(wp);
794     if (!upgrade) break;
795     if (!isDup(upgrade)) choices.push(upgrade); else {
796         // try a different weapon
797         let found = false;
798         for (let tries = 0; tries < 10; tries++) {
799             const wp2 = p.weapons[Math.floor(Math.random() * p.weapons.length)];
800             const ug2 = generateUpgrade(wp2);
801             if (ug2 && !isDup(ug2)) { choices.push(ug2); found = true; break; }
802         }
803         if (!found) break;
804     }
805 }
806
807 return choices;
808 }
809
810 function generateUpgrade(wp) {
811     const type = wp.type;
812     const options = [];
813     // Get themed names for this weapon
```

```

814   const themes = (WEAPON_UPGRADE_THEMES[type] || {})[wp.name] || {};
815   // Helper: get themed name, fall back to generic
816   function tname(key, fallback) {
817     const n = themes[key];
818     if (n === '--') return null; // explicitly disabled for this weapon
819     return n || fallback;
820   }
821
822   // Helper: brighten color on damage boost
823   const brighten = (hex, amt) => {
824     let r = parseInt(hex.slice(1,3),16), g = parseInt(hex.slice(3,5),16), b = parseInt(hex.s
lice(5,7),16);
825     r = Math.min(255, r + amt); g = Math.min(255, g + amt); b = Math.min(255, b + amt);
826     return '#' + [r,g,b].map(v => v.toString(16).padStart(2,'0')).join('');
827   };
828
829   // Common damage upgrade
830   const commonDesc = type === 'projectile' ? '■■■ +1■■■■■■' : type === 'orbital' ? '■■■ +1■■■■■■
' : '■■■ +1■■■■■■';
831   const commonStat = type === 'projectile' ? '■■■+1 ■■■+' : type === 'orbital' ? '■■■+1 ■■■+' :
'■■■+1 ■■■+';
832   const dn = tname('damage', '■■■ ' + wp.name);
833   if (dn) options.push({ name: dn, desc: commonDesc, stat: commonStat, rarity: 'common', eff
ect: w => {
834     w.damage += 1; w.level++;
835     if (w.type === 'projectile') { w.bulletRadius = Math.min(18, w.bulletRadius + 1); w.vfx.
muzzleFlash = true; w.vfx.energyRing = true; }
836     if (w.type === 'orbital') { w.size = Math.min(22, w.size + 1); w.vfx.sparkTrail = true;
}
837     if (w.type === 'field') w.fieldRadius = Math.floor(w.fieldRadius * 1.05);
838     w.color = brighten(w.color, 8);
839   }, icon: '■' });
840
841   if (type === 'projectile') {
842     const mn = tname('multi', wp.name + '.■■■');
843     if (mn) options.push({ name: mn, desc: '■■■■ +1', stat: '■■■+1', rarity: 'common', effect:
w => { w.count += 1; w.vfx.scatterBurst = true; w.level++; }, icon: '◆' });
844     const sn = tname('speed', wp.name + '.■■■');
845     if (sn) options.push({ name: sn, desc: '■■■ -15%', stat: '■■■-15%', rarity: 'common', effe
ct: w => { w.cooldown = Math.max(60, Math.floor(w.cooldown * 0.85)); w.vfx.heatWave = true;
w.level++; }, icon: '■' });
846     if (!wp.piercing) {
847       const pn = tname('pierce', wp.name + '.■■■');
848       if (pn) options.push({ name: pn, desc: '■■■■■■■■■■■■■■■■■■■■', stat: '■■■', rarity: 'rare', effe
ct: w => { w.piercing = true; w.vfx.piercingGlow = true; w.vfx.energyRing = true; w.color =
brighten(w.color, 30); w.level++; }, icon: '►' });
849     }
850     const scn = tname('scatter', wp.name + '.■■■');
851     if (scn) options.push({ name: scn, desc: '■■■■ +2■■■■■■', stat: '■■■+2', rarity: 'rare', ef
fect: w => { w.count += 2; w.spread += 0.06; w.vfx.scatterBurst = true; w.vfx.muzzleFlash =
true; w.level++; }, icon: '◆◆' });
852     const stn = tname('storm', wp.name + '.■■■');
853     if (stn) options.push({ name: stn, desc: '■■■ +2■■■ -20%■■■■■■', stat: '■■■+2 ■■■-20%', rari
ty: 'epic', effect: w => {
854       w.damage += 2; w.cooldown = Math.max(60, Math.floor(w.cooldown * 0.8));
855       w.bulletRadius = Math.min(18, w.bulletRadius + 2);
856       w.vfx.energyRing = true; w.vfx.muzzleFlash = true; w.vfx.heatWave = true;
857       w.color = brighten(w.color, 20); w.level += 2;
858     }, icon: '■' });
859   } else if (type === 'orbital') {
860     const tn = tname('twin', wp.name + '.■■■');
861     if (tn) options.push({ name: tn, desc: '■■■■ +1', stat: '■■■+1', rarity: 'common', effect:
w => { w.count += 1; w.level++; }, icon: '◆' });
862     const rn = tname('range', wp.name + '.■■■');
863     if (rn) options.push({ name: rn, desc: '■■■■ +20%', stat: '■■■+20%', rarity: 'common', ef
fect: w => { w.orbitRadius = Math.floor(w.orbitRadius * 1.2); w.level++; }, icon: '■' });
864     const spn = tname('speed', wp.name + '.■■■');
865     if (spn) options.push({ name: spn, desc: '■■■■ +25%', stat: '■■■+25%', rarity: 'common',
effect: w => { w.orbitSpeed *= 1.25; w.level++; }, icon: '■' });
866     const gn = tname('gravity', wp.name + '.■■■');
867     if (gn) options.push({ name: gn, desc: '■■■ +2■■■ +10%■■■■■■', stat: '■■■+2 ■■■+10%', rarity
: 'rare', effect: w => {
868       w.damage += 2; w.orbitRadius = Math.floor(w.orbitRadius * 1.1);
869       w.size = Math.min(22, w.size + 1); w.vfx.sparkTrail = true; w.color = brighten(w.color
, 15); w.level++;
870     }, icon: '■' });
871     const cln = tname('cluster', wp.name + '.■■■');
872     if (cln) options.push({ name: cln, desc: '■■■■ +2■■■ +1', stat: '■■■+2 ■■■+1', rarity: 'epi

```

```

    c', effect: w => {
873       w.count += 2; w.damage += 1; w.size = Math.min(22, w.size + 1);
874       w.vfx.sparkTrail = true; w.vfx.energyRing = true;
875       w.color = brighten(w.color, 20); w.level += 2;
876     }, icon: '■' });
877   } else if (type === 'field') {
878     const pwn = tname('power', wp.name + '.■■■');
879     if (pwn) options.push({ name: pwn, desc: '■■■ +2', stat: '■■■+2', rarity: 'common', effect
: w => {
880       w.damage += 2; w.color = brighten(w.color, 10); w.level++;
881     }, icon: '■' });
882     const exn = tname('expand', wp.name + '.■■■');
883     if (exn) options.push({ name: exn, desc: '■■■■ +20%', stat: '■■■+20%', rarity: 'common',
effect: w => { w.fieldRadius = Math.floor(w.fieldRadius * 1.2); w.level++; }, icon: '■' });
884     const pn = tname('pulse', wp.name + '.■■■');
885     if (pn) options.push({ name: pn, desc: '■■■■ -30%', stat: '■■■-30%', rarity: 'common', ef
fect: w => { w.cooldown = Math.max(150, Math.floor(w.cooldown * 0.7)); w.level++; }, icon: '
■' });
886     const stn = tname('storm', wp.name + '.■■■');
887     if (stn) options.push({ name: stn, desc: '■■■ +3■■■ +10%', stat: '■■■+3 ■■■+10%', rarity: '
rare', effect: w => {
888       w.damage += 3; w.fieldRadius = Math.floor(w.fieldRadius * 1.1);
889       w.color = brighten(w.color, 20); w.level++;
890     }, icon: '■' });
891     const sin = tname('singularity', wp.name + '.■■■');
892     if (sin) options.push({ name: sin, desc: '■■■ +5■■■ -50■■■ +15%', stat: '■■■+5 ■■■-50%', r
arity: 'epic', effect: w => {
893       w.damage += 5; w.cooldown = Math.max(120, Math.floor(w.cooldown * 0.5));
894       w.fieldRadius = Math.floor(w.fieldRadius * 1.15);
895       w.color = brighten(w.color, 30); w.level += 2;
896     }, icon: '■' });
897   }
898
899   if (options.length === 0) return null;
900
901   // Pick random option, weighted by rarity
902   const weights = { common: 50, rare: 30, epic: 15 };
903   const pool = [];
904   for (const opt of options) {
905     const wgt = weights[opt.rarity] || 10;
906     for (let i = 0; i < wgt; i++) pool.push(opt);
907   }
908   const chosen = pool[Math.floor(Math.random() * pool.length)];
909
910   return {
911     kind: 'upgrade',
912     name: chosen.name,
913     icon: chosen.icon,
914     targetWeapon: wp,
915     desc: chosen.desc,
916     stat: chosen.stat,
917     rarity: chosen.rarity,
918     effect: chosen.effect,
919   };
920 }
921
922 function getTypeDescription(type) {
923   switch (type) {
924     case 'projectile': return '■■■■■■■■■■';
925     case 'orbital': return '■■■■■■■■■■■■■■■■■■■■';
926     case 'field': return '■■■■■■■■■■■■■■■■■■■■0.5■■■■■■■■■■';
927     default: return '';
928   }
929 }
930
931 // =====
932 //  MONSTER TYPES
933 //  =====
934
935 const MONSTER_TYPES = {
936   slime: { hp:5, speed:1.0, radius:13, color:'#c86020', xp:1, glow:4, shape:'blob', unlo
ck:0 },
937   bat: { hp:1, speed:2.2, radius:9, color:'#58b828', xp:1, glow:4, shape:'diamond', unlo
ck:35 },
938   zombie: { hp:7, speed:1.3, radius:15, color:'#58b828', xp:2, glow:4, shape:'square', unlo
ck:75 },
939   ghost: { hp:3, speed:1.8, radius:12, color:'#dddde8', xp:2, glow:4, shape:'circle', unlo
ck:120 },

```

```

940   demon: { hp:8, speed:1.6, radius:16, color:'#e89820', xp:4, glow:4,  shape:'triangle',unl
ock:175 },
941   boss:  { hp:35,speed:0.8, radius:35, color:'#8850b8', xp:10,glow:6,  shape:'star',  unlo
ck:0  },
942 };
943
944 // ---- Game State ----
945 let game;
946 function initGame() {
947   game = {
948     state: 'playing',
949     time: 0, frame: 0,
950     player: {
951       x: CFG.MAP_W / 2, y: CFG.MAP_H / 2,
952       level: 1, xp: 0, killCount: 0, alive: true, trail: [],
953       weapons: [],
954       // Dash
955       dashCooldown: 0, dashFrames: 0, invulnFrames: 0,
956       dashDirX: 0, dashDirY: 0, afterimages: [],
957     },
958     monsters: [], projectiles: [], particles: [], floatTexts: [],
959     obstacles: [],
960     camera: { x: CFG.MAP_W / 2, y: CFG.MAP_H / 2 },
961     shake: 0, spawnTimer: 0, bossTimer: 0, flashTimer: 0, deathAnim: 0,
962     currentPhase: 1, phaseAlert: 0,
963     upgradeChoices: [],
964     upgradeActive: false,
965     paused: false,
966     // Combo
967     combo: 0, comboTimer: 0, comboMax: 0,
968     // Crowns
969     crowns: 0, crownNextAt: 240,
970   };
971   // Start with projectile weapon (fire flower)
972   const starter = createWeapon('projectile');
973   starter.name = '■■■■';
974   starter.damage = 1;
975   starter.cooldown = 900;
976   starter.count = 1;
977   starter.speed = 10;
978   starter.piercing = false;
979   starter.bulletRadius = 5;
980   starter.color = '#ff6600';
981   game.player.weapons.push(starter);
982   game.obstacles = generateObstacles();
983 }
984
985 function togglePause() {
986   if (!game || game.state !== 'playing') return;
987   game.paused = !game.paused;
988   const btn = document.getElementById('btnPause');
989   const overlay = document.getElementById('pauseOverlay');
990   const dashBtn = document.getElementById('dashBtn');
991   if (game.paused) {
992     btn.textContent = '■';
993     overlay.classList.add('show');
994     buildPauseStats();
995     if (dashBtn) dashBtn.style.display = 'none';
996   } else {
997     btn.textContent = '■';
998     overlay.classList.remove('show');
999     if (dashBtn) dashBtn.style.display = '';
1000   }
1001 }
1002
1003 // ---- Dash ----
1004 const DASH_SPEED = 14, DASH_FRAMES = 8, DASH_INVULN = 26, DASH_COOLDOWN = 120; // ~2s at 60f
ps
1005
1006 // ---- Combo ----
1007 const COMBO_TIMEOUT = 180; // 3 seconds at 60fps
1008 function comboMultiplier(combo) {
1009   if (combo >= 30) return 3.0;
1010   if (combo >= 15) return 2.0;
1011   if (combo >= 5) return 1.5;
1012   return 1.0;
1013 }
1014 function tryDash() {

```

```

1015     const p = game.player;
1016     if (p.dashCooldown > 0 || p.dashFrames > 0) return;
1017     // Get direction from current input
1018     let dx = 0, dy = 0;
1019     if (keys['w'] || keys['arrowup']) dy -= 1;
1020     if (keys['s'] || keys['arrowdown']) dy += 1;
1021     if (keys['a'] || keys['arrowleft']) dx -= 1;
1022     if (keys['d'] || keys['arrowright']) dx += 1;
1023     if (joy.active && (Math.abs(joy.dx) > 5 || Math.abs(joy.dy) > 5)) { dx = joy.dx; dy = joy.
dy; }
1024     const len = Math.hypot(dx, dy);
1025     if (len < 0.1) { dx = p._facingRight !== false ? 1 : -1; dy = 0; } // dash in facing dir i
f no input
1026     else { dx /= len; dy /= len; }
1027     p.dashDirX = dx; p.dashDirY = dy;
1028     p.dashFrames = DASH_FRAMES;
1029     p.invulnFrames = DASH_INVULN;
1030     p.dashCooldown = DASH_COOLDOWN;
1031     // Dash burst particles
1032     for (let i = 0; i < 12; i++) {
1033         const a = Math.atan2(dy, dx) + Math.PI + (Math.random() - 0.5) * 1.2;
1034         const s = rand(2, 6);
1035         game.particles.push({ x: p.x, y: p.y, vx: Math.cos(a) * s, vy: Math.sin(a) * s, life: 0.
5, decay: 0.04, size: rand(2, 5), color: '#ffd700' });
1036     }
1037     playSound('dash');
1038 }
1039
1040 function buildPauseStats() {
1041     const container = document.getElementById('pauseStats');
1042     const typeNames = { projectile: '■■■', orbital: '■■■', field: '■■■' };
1043     const typeIcons = { projectile: '♣', orbital: '■', field: '■' };
1044     let html = '';
1045     for (const w of game.player.weapons) {
1046         const abilities = [];
1047         if (w.piercing) abilities.push('■■■');
1048         const typeKey = w.type.replace('projectile', 'projectile').replace('orbital', 'orbital').r
eplace('field', 'field');
1049         html += `<div class="pause-weapon-card" style="border-left:3px solid ${w.color}">
1050             <div class="pw-header">
1051                 <div class="pw-icon">${w.icon}</div>
1052                 <div class="pw-name" style="color:${w.color}">${w.name}</div>
1053                 <div class="pw-lv">Lv.${w.level}</div>
1054             </div>
1055             <div class="pw-type-badge pw-type-${w.type}">${typeIcons[w.type]} || '' ${typeNames[w.
type]} || w.type</div>`;
1056
1057         if (w.type === 'projectile') {
1058             html += `<div class="pw-row"><span>■■■</span><span style="color:${w.color}">${w.damage}
</span></div>`;
1059             html += `<div class="pw-row"><span>■■■■</span><span>${w.count}</span></div>`;
1060             html += `<div class="pw-row"><span>■■■</span><span>${Math.round(w.cooldown)}ms</span></
div>`;
1061             html += `<div class="pw-row"><span>■■■■</span><span>${w.speed}</span></div>`;
1062         } else if (w.type === 'orbital') {
1063             html += `<div class="pw-row"><span>■■■</span><span style="color:${w.color}">${w.damage}
</span></div>`;
1064             html += `<div class="pw-row"><span>■■■</span><span>${w.count}</span></div>`;
1065             html += `<div class="pw-row"><span>■■■■</span><span>${w.orbitRadius}px</span></div>`;
1066             html += `<div class="pw-row"><span>■■■</span><span>${(w.orbitSpeed * 100).toFixed(1)}</
span></div>`;
1067         } else if (w.type === 'field') {
1068             html += `<div class="pw-row"><span>■■■</span><span style="color:${w.color}">${w.damage}
</span></div>`;
1069             html += `<div class="pw-row"><span>■■■</span><span>■${(w.cooldown / 1000).toFixed(1)}s<
/span></div>`;
1070             html += `<div class="pw-row"><span>■■■■</span><span>${w.fieldRadius}px</span></div>`;
1071         }
1072     }
1073
1074     if (abilities.length > 0) {
1075         html += `<div class="pw-abilities">${abilities.map(a => `<span class="pw-abi">${a}</sp
an>`).join('')}</div>`;
1076     }
1077     html += `</div>`;
1078 }
1079 container.innerHTML = html;
1080 }

```



```

1081
1082 function backToMenu() {
1083     stopBgMusic();
1084     game.paused = false;
1085     game.state = 'dead'; // prevent resume / pause from working
1086     document.getElementById('pauseOverlay').classList.remove('show');
1087     document.getElementById('btnPause').textContent = '■';
1088     const dashBtn = document.getElementById('dashBtn');
1089     if (dashBtn) dashBtn.style.display = 'none';
1090     document.getElementById('menu').classList.remove('hidden');
1091     document.getElementById('gameover').classList.remove('show');
1092     showMenuBest();
1093 }
1094
1095 function restartGame() {
1096     game.paused = false;
1097     document.getElementById('pauseOverlay').classList.remove('show');
1098     document.getElementById('btnPause').textContent = '■';
1099     const dashBtn = document.getElementById('dashBtn');
1100     if (dashBtn) dashBtn.style.display = '';
1101     startGame();
1102 }
1103
1104 // ---- Input ----
1105 const keys = {};
1106 window.addEventListener('keydown', e => {
1107     const k = e.key.toLowerCase();
1108     keys[k] = true;
1109     if (k === 'p' && game && game.state === 'playing' && !game.upgradeActive) togglePause();
1110     if (k === ' ' && game && game.state === 'playing' && !game.upgradeActive) { e.preventDefault(); tryDash(); }
1111 });
1112 window.addEventListener('keyup', e => { keys[e.key.toLowerCase()] = false; });
1113
1114 let joy = { active: false, cx: 0, cy: 0, dx: 0, dy: 0, id: null };
1115 function getTouchPos(e, touch) { const rect = canvas.getBoundingClientRect(); return { x: touch.clientX - rect.left, y: touch.clientY - rect.top }; }
1116
1117 canvas.addEventListener('touchstart', e => {
1118     e.preventDefault();
1119     if (game.state !== 'playing') return;
1120     if (game.upgradeActive) { handleUpgradeTap(e); return; }
1121     // Use first available touch as joystick (dash button handles its own touches)
1122     if (joy.active) return;
1123     const touch = e.changedTouches[0], pos = getTouchPos(e, touch);
1124     joy.active = true; joy.id = touch.identifier; joy.cx = pos.x; joy.cy = pos.y; joy.dx = 0; joy.dy = 0;
1125 }, { passive: false });
1126 canvas.addEventListener('touchmove', e => {
1127     e.preventDefault();
1128     if (game.upgradeActive) return;
1129     if (!joy.active) return;
1130     for (const t of e.changedTouches) {
1131         if (t.identifier === joy.id) {
1132             const pos = getTouchPos(e, t), dx = pos.x - joy.cx, dy = pos.y - joy.cy, dist = Math.hypot(dx, dy), maxDist = 60;
1133             if (dist > maxDist) { joy.dx = (dx / dist) * maxDist; joy.dy = (dy / dist) * maxDist; }
1134             else { joy.dx = dx; joy.dy = dy; }
1135             break;
1136         }
1137     }
1138 }, { passive: false });
1139 canvas.addEventListener('touchend', e => {
1140     e.preventDefault();
1141     for (const t of e.changedTouches) {
1142         if (t.identifier === joy.id) { joy.active = false; joy.id = null; joy.dx = 0; joy.dy = 0; break; }
1143     }
1144 }, { passive: false });
1145 canvas.addEventListener('touchcancel', e => {
1146     joy.active = false; joy.id = null; joy.dx = 0; joy.dy = 0;
1147 }, { passive: false });
1148
1149 // ---- Utility ----
1150 const rand = (a, b) => a + Math.random() * (b - a);
1151 const clamp = (v, a, b) => Math.max(a, Math.min(b, v));
1152 const dist2 = (a, b) => { const dx = a.x - b.x, dy = a.y - b.y; return dx * dx + dy * dy; };

```

```

1153 const angleTo = (a, b) => Math.atan2(b.y - a.y, b.x - a.x);
1154
1155 // ---- Spawning ----
1156 function spawnMonster() {
1157     if (game.monsters.length >= CFG.MAX_MONSTERS) return;
1158     const t = game.time;
1159     const available = [];
1160     for (const [key, type] of Object.entries(MONSTER_TYPES)) {
1161         if (key === 'boss') continue;
1162         if (t >= type.unlock) available.push(key);
1163     }
1164     if (available.length === 0) available.push('slime');
1165     const typeName = available[Math.floor(Math.random() * available.length)];
1166     const type = MONSTER_TYPES[typeName];
1167     const angle = Math.random() * Math.PI * 2;
1168     const spawnDist = Math.max(W, H) / 2 + 60;
1169     const x = clamp(game.player.x + Math.cos(angle) * spawnDist, 30, CFG.MAP_W - 30);
1170     const y = clamp(game.player.y + Math.sin(angle) * spawnDist, 30, CFG.MAP_H - 30);
1171
1172     // Elite chance - uses difficulty curve
1173     const eliteChance = diffEliteChance(t);
1174     const isElite = Math.random() < eliteChance && available.length > 1;
1175
1176     const timeHpMult = diffHpMult(t);
1177     const hp = Math.floor((isElite ? type.hp * 3 + 2 : type.hp) * timeHpMult);
1178     const radius = isElite ? type.radius * 1.6 : type.radius;
1179     const xp = isElite ? 5 + Math.floor(Math.random() * 6) : type.xp;
1180     const speedMult = diffSpeedMult(t);
1181     const speedBonus = diffSpeedBonus(t);
1182     const speed = (isElite ? type.speed * 0.85 : type.speed * rand(0.85, 1.15)) * speedMult +
    speedBonus;
1183
1184     game.monsters.push({
1185         type: typeName, x, y, hp, maxHp: hp,
1186         speed, radius,
1187         color: type.color, glow: isElite ? type.glow + 15 : type.glow, shape: type.shape, xp,
1188         flash: 0, spawnAnim: 1, wobble: Math.random() * Math.PI * 2,
1189         hitByOrbit: new Set(), slowTimer: 0,
1190         isElite,
1191     });
1192
1193     if (isElite) {
1194         showFloatText(x, y - 30, '■ ■■■!', '#ffd700', 1.5);
1195         playSound('elite');
1196     }
1197 }
1198
1199 function spawnBoss() {
1200     const type = MONSTER_TYPES.boss;
1201     const angle = Math.random() * Math.PI * 2;
1202     const spawnDist = Math.max(W, H) / 2 + 80;
1203     game.monsters.push({
1204         type: 'boss',
1205         x: clamp(game.player.x + Math.cos(angle) * spawnDist, 50, CFG.MAP_W - 50),
1206         y: clamp(game.player.y + Math.sin(angle) * spawnDist, 50, CFG.MAP_H - 50),
1207         hp: Math.floor(type.hp * (1 + game.time * 0.02)), maxHp: Math.floor(type.hp * (1 + game.
    time * 0.02)), speed: type.speed, radius: type.radius,
1208         color: type.color, glow: type.glow, shape: type.shape, xp: type.xp,
1209         flash: 0, spawnAnim: 1, wobble: 0, isBoss: true, hitByOrbit: new Set(), slowTimer: 0,
1210     });
1211     showFloatText(game.player.x, game.player.y - 50, '■ BOSS ■!', '#ff00ff', 1.5);
1212 }
1213
1214 // ---- Combat ----
1215 function fireProjectile(weapon) {
1216     const p = game.player;
1217     let nearest = null, minD = Infinity;
1218     for (const m of game.monsters) { const d = dist2(p, m); if (d < minD) { minD = d; nearest
    = m; } }
1219     if (!nearest) return;
1220     const baseAngle = angleTo(p, nearest);
1221     const count = weapon.count;
1222     const totalSpread = weapon.spread * (count - 1);
1223
1224     // Muzzle flash particles
1225     if (weapon.vfx.muzzleFlash) {
1226         for (let i = 0; i < 6; i++) {
1227             const a = baseAngle + (Math.random() - 0.5) * 0.8;

```

```

1228     const s = 3 + Math.random() * 5;
1229     game.particles.push({
1230         x: p.x + Math.cos(baseAngle) * CFG.PLAYER_RADIUS,
1231         y: p.y + Math.sin(baseAngle) * CFG.PLAYER_RADIUS,
1232         vx: Math.cos(a) * s, vy: Math.sin(a) * s,
1233         life: 0.3 + Math.random() * 0.2, decay: 0.04,
1234         size: 2 + Math.random() * 3, color: weapon.color,
1235     });
1236     }
1237 }
1238
1239 // Scatter burst extra flash
1240 if (weapon.vfx.scatterBurst && count > 2) {
1241     for (let i = 0; i < 3; i++) {
1242         const a = baseAngle + (Math.random() - 0.5) * 0.5;
1243         game.particles.push({
1244             x: p.x + Math.cos(baseAngle) * CFG.PLAYER_RADIUS,
1245             y: p.y + Math.sin(baseAngle) * CFG.PLAYER_RADIUS,
1246             vx: Math.cos(a) * 2, vy: Math.sin(a) * 2,
1247             life: 0.25, decay: 0.05,
1248             size: 2, color: 'ffffff',
1249         });
1250     }
1251 }
1252
1253 for (let i = 0; i < count; i++) {
1254     const offset = count === 1 ? 0 : (i / (count - 1) - 0.5) * totalSpread * 2;
1255     const ang = baseAngle + offset;
1256     game.projectiles.push({
1257         x: p.x + Math.cos(ang) * CFG.PLAYER_RADIUS,
1258         y: p.y + Math.sin(ang) * CFG.PLAYER_RADIUS,
1259         vx: Math.cos(ang) * weapon.speed,
1260         vy: Math.sin(ang) * weapon.speed,
1261         damage: weapon.damage, radius: weapon.bulletRadius,
1262         color: weapon.color, piercing: weapon.piercing,
1263         life: CFG.PROJECTILE_RANGE / weapon.speed,
1264         hitSet: new Set(), trail: [],
1265         weaponId: weapon.id,
1266         vfx: weapon.vfx, // carry visual effects
1267         vStyle: weapon.vStyle, // carry visual style
1268     });
1269 }
1270 }
1271
1272 function tickField(weapon) {
1273     const p = game.player;
1274     let hitAny = false;
1275     for (let i = game.monsters.length - 1; i >= 0; i--) {
1276         const m = game.monsters[i];
1277         const d = Math.hypot(m.x - p.x, m.y - p.y);
1278         if (d < weapon.fieldRadius + m.radius) {
1279             if (damageMonster(m, { damage: weapon.damage, color: weapon.color, piercing: true, hit
1280 Set: new Set(), vfx: weapon.vfx })) {
1281                 game.monsters.splice(i, 1);
1282             }
1283             // Zap line effect
1284             game.particles.push({
1285                 x: p.x, y: p.y, vx: 0, vy: 0,
1286                 life: 0.3, decay: 0.04, size: 0, color: weapon.color,
1287                 isZap: true, tx: m.x, ty: m.y,
1288             });
1289             hitAny = true;
1290         }
1291     }
1292     // Pulsing ring on tick
1293     game.particles.push({
1294         x: p.x, y: p.y, vx: 0, vy: 0,
1295         life: 0.5, decay: 0.03, size: weapon.fieldRadius,
1296         color: weapon.color, isFieldPulse: true, maxSize: weapon.fieldRadius,
1297     });
1298     if (hitAny) playSound('shoot');
1299 }
1300
1301 function damageMonster(m, proj) {
1302     m.hp -= proj.damage;
1303     m.flash = 8;
1304     const vfx = proj.vfx || {};
1305     const hitCount = vfx.energyRing ? 10 : 6;

```

```

1305     const hitSize = vfx.energyRing ? 4 : 3;
1306     for (let i = 0; i < hitCount; i++) {
1307         const a = Math.random() * Math.PI * 2, s = rand(1, hitSize);
1308         const clr = i % 3 === 0 && vfx.energyRing ? (vfx.piercingGlow ? '#ffffff' : proj.color)
1309         : m.color;
1310         game.particles.push({ x: m.x, y: m.y, vx: Math.cos(a) * s, vy: Math.sin(a) * s, life: 0.
1311         6, decay: 0.03, size: rand(2, 5), color: clr });
1312     }
1313     // Energy burst on piercing hit
1314     if (vfx.piercingGlow) {
1315         for (let i = 0; i < 4; i++) {
1316             const a = Math.random() * Math.PI * 2;
1317             game.particles.push({ x: m.x, y: m.y, vx: Math.cos(a) * 2, vy: Math.sin(a) * 2, life:
1318             0.4, decay: 0.05, size: 2, color: '#ffffff' });
1319         }
1320     }
1321     if (m.hp <= 0) {
1322         const count = m.isBoss ? 60 : (m.isElite ? 35 : 15);
1323         for (let i = 0; i < count; i++) {
1324             const a = Math.random() * Math.PI * 2, s = rand(2, m.isBoss ? 9 : 5);
1325             const clr = m.isElite ? (i % 2 ? '#ffd700' : '#ff8aa') : (i % 2 ? m.color : '#ffffff'
1326             );
1327             game.particles.push({ x: m.x, y: m.y, vx: Math.cos(a) * s, vy: Math.sin(a) * s, life:
1328             1, decay: 0.018, size: rand(3, m.isBoss ? 10 : 6), color: clr });
1329         }
1330         game.player.killCount++;
1331         // Combo system
1332         game.combo++;
1333         game.comboTimer = COMBO_TIMEOUT;
1334         if (game.combo > game.comboMax) game.comboMax = game.combo;
1335         const mult = comboMultiplier(game.combo);
1336         const xpGain = Math.round(m.xp * mult);
1337         gainXP(m.xp);
1338         showFloatText(m.x, m.y, '+' + xpGain + ' XP', mult > 1 ? '#ffaa00' : '#00ff88', 0.8);
1339         // Combo milestone notification
1340         if (game.combo === 5 || game.combo === 15 || game.combo === 30) {
1341             showFloatText(m.x, m.y - 25, `${game.combo}■■! ×${mult.toFixed(1)}`, '#ffaa00', 1.2);
1342         }
1343         if (m.isBoss) { game.shake = 12; showFloatText(m.x, m.y - 30, 'BOSS ■■!', '#ff00ff', 1.2
1344         ); }
1345         playSound(m.isBoss ? 'bosskill' : 'kill');
1346         return true;
1347     }
1348     return false;
1349 }
1350
1351 function gainXP(amount) {
1352     const p = game.player;
1353     if (game.upgradeActive) return; // Don't process XP during upgrade selection
1354     const mult = comboMultiplier(game.combo);
1355     const final = Math.round(amount * mult);
1356     p.xp += final;
1357     while (p.xp >= xpNeeded(p.level)) {
1358         p.xp -= xpNeeded(p.level);
1359         p.level++;
1360         onLevelUp();
1361     }
1362 }
1363
1364 function onLevelUp() {
1365     const p = game.player;
1366     // Golden burst
1367     for (let i = 0; i < 40; i++) {
1368         const a = Math.random() * Math.PI * 2, s = rand(2, 7);
1369         game.particles.push({ x: p.x, y: p.y, vx: Math.cos(a) * s, vy: Math.sin(a) * s, life: 1,
1370         decay: 0.015, size: rand(3, 7), color: i % 2 ? '#ffd700' : '#ff8aa' });
1371     }
1372     game.flashTimer = 20;
1373     game.shake = 6;
1374     playSound('levelup');
1375
1376     // Generate and show upgrade choices
1377     game.upgradeChoices = generateUpgradeChoices();
1378     game.upgradeActive = true;
1379     showUpgradeModal(game.upgradeChoices);
1380 }
1381
1382 function applyUpgrade(choice) {

```

```

1376     if (choice.kind === 'new') {
1377         game.player.weapons.push(choice.weapon);
1378         showToast(choice.name, '■■■■■: ' + WEAPON_BASES[choice.type].name + '■');
1379     } else if (choice.kind === 'upgrade') {
1380         choice.effect(choice.targetWeapon);
1381         choice.targetWeapon.level++;
1382         showToast(choice.name, choice.stat);
1383     }
1384     game.upgradeActive = false;
1385     hideUpgradeModal();
1386     playSound('levelup');
1387 }
1388
1389 // ---- Upgrade Modal ----
1390 const upgradeModal = document.getElementById('upgradeModal');
1391 const upgradeChoicesEl = document.getElementById('upgradeChoices');
1392
1393 function showUpgradeModal(choices) {
1394     upgradeChoicesEl.innerHTML = '';
1395     for (let i = 0; i < choices.length; i++) {
1396         const c = choices[i];
1397         const card = document.createElement('div');
1398         card.className = `upgrade-card ${c.rarity}`;
1399         card.innerHTML = `
1400             <div class="card-rarity">${rarityLabel(c.rarity)}</div>
1401             <div class="card-icon">${c.icon}</div>
1402             <div class="card-name">${c.name}</div>
1403             <div class="card-desc">${c.desc}</div>
1404             ${c.stat ? `<div class="card-stat">${c.stat}</div>` : ''}
1405         `;
1406         card.addEventListener('click', () => applyUpgrade(c));
1407         card.addEventListener('touchend', (e) => { e.preventDefault(); applyUpgrade(c); });
1408         upgradeChoicesEl.appendChild(card);
1409     }
1410     upgradeModal.classList.add('show');
1411     const dashBtn = document.getElementById('dashBtn');
1412     if (dashBtn) dashBtn.style.display = 'none';
1413 }
1414
1415 function hideUpgradeModal() {
1416     upgradeModal.classList.remove('show');
1417     const dashBtn = document.getElementById('dashBtn');
1418     if (dashBtn && game && game.state === 'playing') dashBtn.style.display = '';
1419 }
1420
1421 function handleUpgradeTap(e) {
1422     // Touch handled by individual card listeners
1423 }
1424
1425 function rarityLabel(r) {
1426     switch (r) {
1427         case 'common': return '■■■■■';
1428         case 'rare': return '■■■■■';
1429         case 'epic': return '■■■■■';
1430         case 'legendary': return '■■■■■';
1431         default: return '';
1432     }
1433 }
1434
1435 // ---- Float Text ----
1436 function showFloatText(x, y, text, color, life) {
1437     game.floatTexts.push({ x, y, text, color, life, maxLife: life, vy: -1 });
1438 }
1439
1440 // ---- Toast ----
1441 const toastEl = document.getElementById('toast');
1442 let toastTimer = null;
1443 function showToast(title, sub) {
1444     toastEl.innerHTML = `<div>${title}</div><div class="toast-sub">${sub}</div>`;
1445     toastEl.classList.add('show');
1446     clearTimeout(toastTimer);
1447     toastTimer = setTimeout(() => toastEl.classList.remove('show'), 1800);
1448 }
1449
1450 // ---- Sound ----
1451 let audioCtx = null;
1452 function initAudio() { if (!audioCtx) { try { audioCtx = new (window.AudioContext || window.
    webkitAudioContext)(); } catch (e) {} } }

```

```

1453 function playSound(type) {
1454     if (!audioCtx) return;
1455     const now = audioCtx.currentTime;
1456     const osc = audioCtx.createOscillator(), gain = audioCtx.createGain();
1457     osc.connect(gain); gain.connect(audioCtx.destination);
1458     switch (type) {
1459         case 'shoot': osc.type = 'square'; osc.frequency.setValueAtTime(220, now); osc.frequency
        .exponentialRampToValueAtTime(80, now + 0.08); gain.gain.setValueAtTime(0.03, now); gain.gai
        n.exponentialRampToValueAtTime(0.001, now + 0.08); osc.start(now); osc.stop(now + 0.08); bre
        ak;
1460         case 'kill': osc.type = 'sawtooth'; osc.frequency.setValueAtTime(150, now); osc.frequenc
        y.exponentialRampToValueAtTime(50, now + 0.1); gain.gain.setValueAtTime(0.05, now); gain.gai
        n.exponentialRampToValueAtTime(0.001, now + 0.1); osc.start(now); osc.stop(now + 0.1); break
        ;
1461         case 'bosskill': osc.type = 'sawtooth'; osc.frequency.setValueAtTime(300, now); osc.frequ
        uency.exponentialRampToValueAtTime(40, now + 0.4); gain.gain.setValueAtTime(0.1, now); gain.
        gain.exponentialRampToValueAtTime(0.001, now + 0.4); osc.start(now); osc.stop(now + 0.4); br
        eak;
1462         case 'levelup': [0, 0.08, 0.16].forEach((delay, i) => { const o = audioCtx.createOscilla
        tor(), g = audioCtx.createGain(); o.connect(g); g.connect(audioCtx.destination); o.type = 's
        ine'; o.frequency.setValueAtTime(523 * (i + 1), now + delay); g.gain.setValueAtTime(0.08, no
        w + delay); g.gain.exponentialRampToValueAtTime(0.001, now + delay + 0.15); o.start(now + de
        lay); o.stop(now + delay + 0.15); }); return;
1463         case 'death': osc.type = 'sawtooth'; osc.frequency.setValueAtTime(200, now); osc.frequen
        cy.exponentialRampToValueAtTime(30, now + 0.6); gain.gain.setValueAtTime(0.15, now); gain.ga
        in.exponentialRampToValueAtTime(0.001, now + 0.6); osc.start(now); osc.stop(now + 0.6); brea
        k;
1464         case 'dash': osc.type = 'sine'; osc.frequency.setValueAtTime(800, now); osc.frequency.ex
        ponentialRampToValueAtTime(200, now + 0.15); gain.gain.setValueAtTime(0.06, now); gain.gain.
        exponentialRampToValueAtTime(0.001, now + 0.15); osc.start(now); osc.stop(now + 0.15); break
        ;
1465         case 'elite': osc.type = 'sawtooth'; osc.frequency.setValueAtTime(440, now); osc.frequen
        cy.linearRampToValueAtTime(880, now + 0.3); gain.gain.setValueAtTime(0.08, now); gain.gain.e
        xponentialRampToValueAtTime(0.001, now + 0.3); osc.start(now); osc.stop(now + 0.3); break;
1466     }
1467 }
1468
1469 // ---- Background Music (Mario-style chiptune) ----
1470 let bgMusicActive = false;
1471 let bgMusicNodes = null;
1472
1473 const MELODY_NOTES = [
1474     // Melody: E4 E4 . E4 . C4 E4 G4 . G3 . C4
1475     329.63, 329.63, 0, 329.63, 0, 261.63, 329.63, 392.00,
1476     0, 196.00, 0, 261.63,
1477     // A4 A4 . A4 . G4 E4 C4 . G3 . E4
1478     440.00, 440.00, 0, 440.00, 0, 392.00, 329.63, 261.63,
1479     0, 196.00, 0, 329.63,
1480     // E4 G4 A4 B4 A4 G4 E4 C4 D4 E4 . .
1481     329.63, 392.00, 440.00, 493.88, 440.00, 392.00, 329.63, 261.63,
1482     293.66, 329.63, 0, 0,
1483     // G4 A4 G4 E4 C4 D4 C4 . . . .
1484     392.00, 440.00, 392.00, 329.63, 261.63, 293.66, 261.63, 0,
1485     0, 0, 0, 0,
1486 ];
1487 const MELODY_STEP = 0.14; // seconds per note
1488
1489 function scheduleMelody(startTime) {
1490     const gain = audioCtx.createGain();
1491     gain.gain.value = 0.05;
1492     gain.connect(audioCtx.destination);
1493
1494     let scheduled = [];
1495     for (let i = 0; i < MELODY_NOTES.length; i++) {
1496         const freq = MELODY_NOTES[i];
1497         const t = startTime + i * MELODY_STEP;
1498         if (freq > 0) {
1499             const osc = audioCtx.createOscillator();
1500             osc.type = 'square';
1501             osc.frequency.value = freq;
1502             const noteGain = audioCtx.createGain();
1503             noteGain.gain.setValueAtTime(0.06, t);
1504             noteGain.gain.setValueAtTime(0, t + MELODY_STEP * 0.85);
1505             osc.connect(noteGain); noteGain.connect(gain);
1506             osc.start(t); osc.stop(t + MELODY_STEP);
1507             scheduled.push(osc);
1508         }
1509     }

```

```

1510     return scheduled;
1511 }
1512
1513 function scheduleBass(startTime) {
1514     const gain = audioCtx.createGain();
1515     gain.gain.value = 0.04;
1516     gain.connect(audioCtx.destination);
1517
1518     // Bass pattern: quarter notes on beat
1519     const bassNotes = [130.81, 130.81, 164.81, 164.81, 146.83, 146.83, 130.81, 130.81]; // C3,
C3, E3, E3, D3, D3, C3, C3
1520     const bassStep = MELODY_STEP * 6; // every 6 melody steps = 1 bass note
1521     let scheduled = [];
1522     for (let i = 0; i < 8; i++) {
1523         const t = startTime + i * bassStep;
1524         const osc = audioCtx.createOscillator();
1525         osc.type = 'triangle';
1526         osc.frequency.value = bassNotes[i];
1527         const noteGain = audioCtx.createGain();
1528         noteGain.gain.setValueAtTime(0.04, t);
1529         noteGain.gain.exponentialRampToValueAtTime(0.001, t + bassStep * 0.8);
1530         osc.connect(noteGain); noteGain.connect(gain);
1531         osc.start(t); osc.stop(t + bassStep);
1532         scheduled.push(osc);
1533     }
1534     return scheduled;
1535 }
1536
1537 let bgMusicInterval = null;
1538 function startBgMusic() {
1539     if (!audioCtx || bgMusicActive) return;
1540     bgMusicActive = true;
1541     document.getElementById('btnMusic').textContent = '■';
1542
1543     const totalLen = MELODY_NOTES.length * MELODY_STEP;
1544
1545     function playLoop() {
1546         if (!bgMusicActive) return;
1547         const now = audioCtx.currentTime;
1548         scheduleMelody(now);
1549         scheduleBass(now);
1550     }
1551
1552     playLoop();
1553     bgMusicInterval = setInterval(playLoop, totalLen * 1000);
1554 }
1555
1556 function stopBgMusic() {
1557     bgMusicActive = false;
1558     document.getElementById('btnMusic').textContent = '■';
1559     if (bgMusicInterval) { clearInterval(bgMusicInterval); bgMusicInterval = null; }
1560 }
1561
1562 function toggleMusic() {
1563     if (bgMusicActive) stopBgMusic();
1564     else startBgMusic();
1565 }
1566
1567 function update() {
1568     if (game.upgradeActive) return; // Freeze game during upgrade selection
1569     if (game.paused) return; // Freeze game when paused
1570
1571     if (game.state !== 'playing') {
1572         if (game.state === 'dying') {
1573             game.deathAnim++; updateParticles(); game.shake *= 0.9;
1574             if (game.deathAnim > 50) { game.state = 'gameover'; showGameOver(); }
1575         }
1576         return;
1577     }
1578
1579     game.time += 1 / 60; game.frame++;
1580
1581     // Phase transition alerts
1582     if (game.time > 80 && game.currentPhase === 1) {
1583         game.currentPhase = 2; game.phaseAlert = 120;
1584         showToast('■■■■■ · ■■■■■', '■■■■■■■■■■■■■■■■■■■■');
1585     }
1586     if (game.time > 200 && game.currentPhase === 2) {
1587         game.currentPhase = 3; game.phaseAlert = 120;

```

```

1587     showToast('■■■■ . ■■■■', '■■■■■■■■■■■■');
1588 }
1589
1590 // Crown rewards – after 4 minutes, every 5s
1591 if (game.time >= 240 && game.time >= game.crownNextAt) {
1592     game.crowns++;
1593     game.crownNextAt += 5;
1594     showFloatText(game.player.x, game.player.y - 30, '■ +1', '#ffd700', 1.6);
1595 }
1596
1597 // ---- Player Movement ----
1598 const p = game.player;
1599 let mx = 0, my = 0;
1600 if (keys['w'] || keys['arrowup']) my -= 1;
1601 if (keys['s'] || keys['arrowdown']) my += 1;
1602 if (keys['a'] || keys['arrowleft']) mx -= 1;
1603 if (keys['d'] || keys['arrowright']) mx += 1;
1604 if (joy.active && (Math.abs(joy.dx) > 5 || Math.abs(joy.dy) > 5)) {
1605     const jDist = Math.hypot(joy.dx, joy.dy); mx = joy.dx / jDist; my = joy.dy / jDist;
1606 }
1607 const mLen = Math.hypot(mx, my);
1608
1609 // ---- Dash movement (overrides normal movement) ----
1610 if (p.dashFrames > 0) {
1611     p.x += p.dashDirX * DASH_SPEED;
1612     p.y += p.dashDirY * DASH_SPEED;
1613     p.x = clamp(p.x, CFG.PLAYER_RADIUS, CFG.MAP_W - CFG.PLAYER_RADIUS);
1614     p.y = clamp(p.y, CFG.PLAYER_RADIUS, CFG.MAP_H - CFG.PLAYER_RADIUS);
1615     pushOutOfObstacles(p, CFG.PLAYER_RADIUS);
1616     // Afterimage
1617     p.afterimages.push({ x: p.x, y: p.y, life: 1 });
1618     if (p.afterimages.length > 8) p.afterimages.shift();
1619     p.dashFrames--;
1620 } else if (mLen > 0) {
1621     mx /= mLen; my /= mLen;
1622     p.x += mx * CFG.PLAYER_SPEED; p.y += my * CFG.PLAYER_SPEED;
1623     p.x = clamp(p.x, CFG.PLAYER_RADIUS, CFG.MAP_W - CFG.PLAYER_RADIUS);
1624     p.y = clamp(p.y, CFG.PLAYER_RADIUS, CFG.MAP_H - CFG.PLAYER_RADIUS);
1625     pushOutOfObstacles(p, CFG.PLAYER_RADIUS);
1626     if (game.frame % 3 === 0) { p.trail.push({ x: p.x, y: p.y, life: 1 }); if (p.trail.length > 12) p.trail.shift(); }
1627 }
1628 // Decrement dash timers
1629 if (p.dashCooldown > 0) p.dashCooldown--;
1630 if (p.invulnFrames > 0) p.invulnFrames--;
1631 // Fade afterimages
1632 for (const ai of p.afterimages) ai.life -= 0.12;
1633 p.afterimages = p.afterimages.filter(ai => ai.life > 0);
1634
1635 for (const t of p.trail) t.life -= 0.08;
1636 p.trail = p.trail.filter(t => t.life > 0);
1637
1638 // ---- Camera ----
1639 const targetCamX = clamp(p.x - W / 2, 0, CFG.MAP_W - W);
1640 const targetCamY = clamp(p.y - H / 2, 0, CFG.MAP_H - H);
1641 game.camera.x += (targetCamX - game.camera.x) * 0.12;
1642 game.camera.y += (targetCamY - game.camera.y) * 0.12;
1643
1644 // ---- Weapon Timers ----
1645 const dt = 1000 / 60;
1646 for (const w of p.weapons) {
1647     w.timer -= dt;
1648     if (w.timer <= 0) {
1649         if (w.type === 'projectile') { fireProjectile(w); playSound('shoot'); }
1650         if (w.type === 'field') tickField(w);
1651         w.timer = w.cooldown;
1652     }
1653     // Update orbital angles
1654     if (w.type === 'orbital') {
1655         w.angle += w.orbitSpeed;
1656     }
1657 }
1658
1659 // ---- Spawning (difficulty-scaled) ----
1660 game.spawnTimer -= 1;
1661 if (game.spawnTimer <= 0) {
1662     const t = game.time;
1663     const interval = diffSpawnInterval(t);

```



```

1664     const batch = diffSpawnBatch(t);
1665     for (let i = 0; i < batch; i++) spawnMonster();
1666     game.spawnTimer = interval;
1667 }
1668 game.bossTimer += 1;
1669 const bossInterval = diffBossInterval(game.time) * 60;
1670 if (game.bossTimer > bossInterval) { spawnBoss(); game.bossTimer = 0; }
1671
1672 // ---- Monsters ----
1673 let nearbyCount = 0;
1674 // Clear orbital hit tracking for new frame
1675 for (const m of game.monsters) m.hitByOrbit = new Set();
1676
1677 for (let i = game.monsters.length - 1; i >= 0; i--) {
1678     const m = game.monsters[i];
1679     m.wobble += 0.1;
1680     if (m.spawnAnim > 0) m.spawnAnim -= 0.04;
1681     if (m.flash > 0) m.flash--;
1682     if (m.slowTimer > 0) m.slowTimer--;
1683     const slowMult = m.slowTimer > 0 ? 0.35 : 1;
1684     const ang = angleTo(m, p);
1685     const wobbleAng = ang + Math.sin(m.wobble) * 0.15;
1686     m.x += Math.cos(wobbleAng) * m.speed * slowMult;
1687     m.y += Math.sin(wobbleAng) * m.speed * slowMult;
1688     m.x = clamp(m.x, m.radius, CFG.MAP_W - m.radius);
1689     m.y = clamp(m.y, m.radius, CFG.MAP_H - m.radius);
1690     pushOutOfObstacles(m, m.radius);
1691
1692     // Player collision (1 HP insta-death, but invulnerable during dash i-frames)
1693     const dx = m.x - p.x, dy = m.y - p.y, d = Math.hypot(dx, dy);
1694     if (d < m.radius + CFG.PLAYER_RADIUS - 2 && m.spawnAnim <= 0 && p.invulnFrames <= 0) { p
layerDeath(); return; }
1695     if (d < Math.max(W, H)) nearbyCount++;
1696
1697     // Orbital weapon collision
1698     for (const w of p.weapons) {
1699         if (w.type !== 'orbital') continue;
1700         if (m.hitByOrbit.has(w.id)) continue;
1701         for (let j = 0; j < w.count; j++) {
1702             const orbAngle = w.angle + (j / w.count) * Math.PI * 2;
1703             const ox = p.x + Math.cos(orbAngle) * w.orbitRadius;
1704             const oy = p.y + Math.sin(orbAngle) * w.orbitRadius;
1705             const od = Math.hypot(ox - m.x, oy - m.y);
1706             if (od < m.radius + w.size) {
1707                 if (damageMonster(m, { damage: w.damage, color: w.color, piercing: true, hitSet: n
ew Set(), vfx: w.vfx })) {
1708                     game.monsters.splice(i, 1);
1709                 }
1710                 m.hitByOrbit.add(w.id);
1711                 break;
1712             }
1713         }
1714         if (i >= game.monsters.length) break;
1715     }
1716 }
1717
1718 // ---- Projectiles ----
1719 for (let i = game.projectiles.length - 1; i >= 0; i--) {
1720     const pr = game.projectiles[i];
1721     pr.x += pr.vx; pr.y += pr.vy; pr.life--;
1722     if (game.frame % 2 === 0) { pr.trail.push({ x: pr.x, y: pr.y, life: 1 }); if (pr.trail.l
ength > 6) pr.trail.shift(); }
1723     for (const t of pr.trail) t.life -= 0.15;
1724     pr.trail = pr.trail.filter(t => t.life > 0);
1725     if (pr.life <= 0 || pr.x < 0 || pr.x > CFG.MAP_W || pr.y < 0 || pr.y > CFG.MAP_H) { game
.projectiles.splice(i, 1); continue; }
1726     let removed = false;
1727     for (let j = game.monsters.length - 1; j >= 0; j--) {
1728         const m = game.monsters[j];
1729         if (pr.hitSet.has(m)) continue;
1730         const dd = Math.hypot(pr.x - m.x, pr.y - m.y);
1731         if (dd < m.radius + pr.radius) {
1732             if (damageMonster(m, pr)) { game.monsters.splice(j, 1); }
1733             else { pr.hitSet.add(m); }
1734             if (!pr.piercing) { game.projectiles.splice(i, 1); removed = true; break; }
1735         }
1736     }
1737 }

```

```

1738
1739 // ---- Particles ----
1740 updateParticles();
1741
1742 // ---- Float Texts ----
1743 for (let i = game.floatTexts.length - 1; i >= 0; i--) {
1744     const ft = game.floatTexts[i];
1745     ft.y += ft.vy; ft.vy *= 0.95; ft.life -= 0.015;
1746     if (ft.life <= 0) game.floatTexts.splice(i, 1);
1747 }
1748
1749 // ---- Effects ----
1750 game.shake *= 0.85;
1751 if (game.flashTimer > 0) game.flashTimer--;
1752
1753 // ---- Combo Timer ----
1754 if (game.comboTimer > 0) {
1755     game.comboTimer--;
1756     if (game.comboTimer <= 0) game.combo = 0;
1757 }
1758
1759 // ---- HUD ----
1760 updateHUD(nearbyCount);
1761 }
1762
1763 function updateParticles() {
1764     for (let i = game.particles.length - 1; i >= 0; i--) {
1765         const pt = game.particles[i];
1766         pt.x += pt.vx; pt.y += pt.vy; pt.vx *= 0.94; pt.vy *= 0.94;
1767         pt.life -= pt.decay;
1768         if (pt.life <= 0) game.particles.splice(i, 1);
1769     }
1770     if (game.particles.length > CFG.MAX_PARTICLES) {
1771         game.particles.splice(0, game.particles.length - CFG.MAX_PARTICLES);
1772     }
1773 }
1774
1775 function playerDeath() {
1776     game.state = 'dying'; game.shake = 25;
1777     const p = game.player;
1778     for (let i = 0; i < 80; i++) {
1779         const a = Math.random() * Math.PI * 2, s = rand(2, 10);
1780         game.particles.push({ x: p.x, y: p.y, vx: Math.cos(a) * s, vy: Math.sin(a) * s, life: 1.
2, decay: 0.012, size: rand(3, 9), color: i % 3 === 0 ? '#ff4466' : (i % 3 === 1 ? '#ffaa00'
: '#ffffff') });
1781     }
1782     playSound('death');
1783 }
1784
1785 // ---- HUD ----
1786 const hudLevel = document.getElementById('hudLevel');
1787 const xpFill = document.getElementById('xpFill');
1788 const hudKills = document.getElementById('hudKills');
1789 const hudTime = document.getElementById('hudTime');
1790 const hudMonsters = document.getElementById('hudMonsters');
1791 const hudWeaponsEl = document.getElementById('hudWeapons');
1792
1793 function updateHUD(nearby) {
1794     const p = game.player;
1795     hudLevel.textContent = `Lv.${p.level}`;
1796     const need = xpNeeded(p.level);
1797     xpFill.style.width = (p.xp / need * 100) + '%';
1798     document.getElementById('hudXp').textContent = `${p.xp} / ${need}`;
1799     hudKills.textContent = `■■ ${p.killCount}`;
1800     const min = Math.floor(game.time / 60), sec = Math.floor(game.time % 60);
1801     hudTime.textContent = `${min}:${sec.toString().padStart(2, '0')}`;
1802     hudMonsters.textContent = `■■ ${nearby} ■`;
1803
1804     // Crown display – show once nearing 4 min or after earning
1805     const crownEl = document.getElementById('hudCrowns');
1806     if (game.time >= 180 || game.crowns > 0) {
1807         crownEl.style.display = '';
1808         crownEl.textContent = `■ ${game.crowns}`;
1809     } else {
1810         crownEl.style.display = 'none';
1811     }
1812
1813     // Dash status

```

```

1814 const dashEl = document.getElementById('hudDash');
1815 const dashBtn = document.getElementById('dashBtn');
1816 if (p.dashCooldown > 0) {
1817     dashEl.classList.add('cooling'); dashEl.classList.remove('ready');
1818     dashBtn.classList.add('cooling'); dashBtn.classList.remove('ready');
1819     // Cooldown ring: fill from 0 to 360 as cooldown progresses
1820     const cdPct = p.dashCooldown / 120; // fraction of cooldown remaining
1821     dashBtn.style.setProperty('--cd', (cdPct * 360) + 'deg');
1822 } else {
1823     dashEl.classList.remove('cooling'); dashEl.classList.add('ready');
1824     dashBtn.classList.remove('cooling'); dashBtn.classList.add('ready');
1825     dashBtn.style.setProperty('--cd', '0deg');
1826 }
1827
1828 // Combo display
1829 const comboEl = document.getElementById('hudCombo');
1830 if (game.combo >= 3) {
1831     const mult = comboMultiplier(game.combo);
1832     comboEl.style.display = '';
1833     comboEl.textContent = `${game.combo} ■■■ x${mult.toFixed(1)}`;
1834 } else {
1835     comboEl.style.display = 'none';
1836 }
1837
1838 // Weapon badges
1839 let wpHtml = '';
1840 for (const w of p.weapons) {
1841     wpHtml += `<span class="hud-wp" style="background:${w.color}22;color:${w.color};border:1
px solid ${w.color}44">${w.icon} ${w.name} Lv${w.level}</span>`;
1842 }
1843 hudWeaponsEl.innerHTML = wpHtml;
1844 }
1845
1846 function showGameOver() {
1847     const p = game.player;
1848     const min = Math.floor(game.time / 60), sec = Math.floor(game.time % 60);
1849     document.getElementById('goTime').textContent = `${min}:${sec.toString().padStart(2, '0')}`;
1850
1851     document.getElementById('goKills').textContent = p.killCount;
1852     document.getElementById('goLevel').textContent = p.level;
1853     document.getElementById('goCombo').textContent = game.comboMax;
1854     // Crowns
1855     if (game.crowns > 0) {
1856         document.getElementById('goCrownStat').style.display = '';
1857         document.getElementById('goCrowns').textContent = game.crowns;
1858     } else {
1859         document.getElementById('goCrownStat').style.display = 'none';
1860     }
1861     dashBtnEl.style.display = 'none';
1862     stopBgMusic();
1863
1864     // Best records
1865     const best = loadBest();
1866     const timeSec = Math.floor(game.time);
1867     const newRec = [];
1868     if (timeSec > best.time) { best.time = timeSec; newRec.push('■■■■'); }
1869     if (p.killCount > best.kills) { best.kills = p.killCount; newRec.push('■■■■'); }
1870     if (p.level > best.level) { best.level = p.level; newRec.push('■■'); }
1871     if (game.comboMax > best.combo) { best.combo = game.comboMax; newRec.push('■■'); }
1872     if (game.crowns > (best.crowns || 0)) { best.crowns = game.crowns; newRec.push('■■'); }
1873     saveBest(best);
1874
1875     const recEl = document.getElementById('goNewRec');
1876     if (newRec.length > 0) {
1877         recEl.style.display = '';
1878         recEl.textContent = `★ ■■■: ${newRec.join(' · ')} ★`;
1879     } else {
1880         recEl.style.display = 'none';
1881     }
1882     const bMin = Math.floor(best.time / 60), bSec = best.time % 60;
1883     document.getElementById('goBest').textContent = `■■: ${bMin}:${bSec.toString().padStart(2,
'0')}` · ${best.kills}■ · Lv.${best.level} · ${best.combo}■ · ${best.crowns||0}■`;
1884     document.getElementById('gameover').classList.add('show');
1885 }
1886
1887 // ---- Best Records (localStorage) ----
1888 function loadBest() {
1889     try {

```

```

1889     const raw = localStorage.getItem('survivalHunter_best');
1890     if (raw) return JSON.parse(raw);
1891   } catch (e) {}
1892   return { time: 0, kills: 0, level: 1, combo: 0, crowns: 0 };
1893 }
1894 function saveBest(b) {
1895   try { localStorage.setItem('survivalHunter_best', JSON.stringify(b)); } catch (e) {}
1896 }
1897 function showMenuBest() {
1898   const best = loadBest();
1899   const el = document.getElementById('menuBest');
1900   if (best.time > 0 || best.kills > 0) {
1901     const min = Math.floor(best.time / 60), sec = best.time % 60;
1902     el.style.display = '';
1903     el.innerHTML = `■ ■■■■<br>${min}:${sec.toString().padStart(2, '0')} · ${best.kills}■ · Lv
1904     .${best.level} · ${best.combo}■■ · ${best.crowns||0}■`;
1905   } else {
1906     el.style.display = 'none';
1907   }
1908 }
1909 // ---- Render ----
1910 function render() {
1911   const sx = (Math.random() - 0.5) * game.shake, sy = (Math.random() - 0.5) * game.shake;
1912   ctx.save();
1913   ctx.translate(-game.camera.x + sx, -game.camera.y + sy);
1914
1915   drawMap();
1916
1917   // Map boundary - brick border
1918   ctx.strokeStyle = 'rgba(139, 69, 19, 0.3)'; ctx.lineWidth = 4;
1919   ctx.strokeRect(0, 0, CFG.MAP_W, CFG.MAP_H);
1920   ctx.strokeStyle = 'rgba(139, 69, 19, 0.15)'; ctx.lineWidth = 2;
1921   ctx.strokeRect(-20, -20, CFG.MAP_W + 40, CFG.MAP_H + 40);
1922
1923   // Particles
1924   for (const pt of game.particles) {
1925     ctx.globalAlpha = pt.life;
1926     if (pt.isZap) {
1927       ctx.strokeStyle = pt.color; ctx.lineWidth = 3; ctx.lineCap = 'round';
1928       ctx.beginPath();
1929       ctx.moveTo(pt.x, pt.y);
1930       const segs = 3, dx = (pt.tx - pt.x) / segs, dy = (pt.ty - pt.y) / segs;
1931       for (let s = 1; s < segs; s++) {
1932         ctx.lineTo(pt.x + dx * s + (Math.random() - 0.5) * 8, pt.y + dy * s + (Math.random()
1933         - 0.5) * 8);
1934       }
1935       ctx.lineTo(pt.tx, pt.ty);
1936       ctx.stroke();
1937     } else if (pt.isFieldPulse) {
1938       const prog = 1 - pt.life / 0.5;
1939       ctx.strokeStyle = pt.color; ctx.lineWidth = 2 * (1 - prog);
1940       const r = pt.maxSize * (0.9 + prog * 0.1);
1941       ctx.beginPath(); ctx.arc(pt.x, pt.y, r, 0, Math.PI * 2); ctx.stroke();
1942     } else if (pt.isPulse) {
1943       // Square expanding rings
1944       const progress = 1 - pt.life;
1945       const r = pt.maxSize * progress;
1946       const lw = (pt.lineWidth || 4) * (1 - progress);
1947
1948       if (pt.isRipple) {
1949         ctx.strokeStyle = pt.color + '66'; ctx.lineWidth = lw;
1950         ctx.strokeRect(pt.x - r, pt.y - r, r * 2, r * 2);
1951       } else {
1952         ctx.strokeStyle = pt.color; ctx.lineWidth = lw;
1953         ctx.strokeRect(pt.x - r, pt.y - r, r * 2, r * 2);
1954       }
1955     } else {
1956       ctx.fillStyle = pt.color;
1957       ctx.fillRect(pt.x - pt.size * pt.life / 2, pt.y - pt.size * pt.life / 2, pt.size * pt.
1958       life, pt.size * pt.life);
1959     }
1960   }
1961   ctx.globalAlpha = 1;
1962
1963   // Float texts
1964   for (const ft of game.floatTexts) {
1965     ctx.globalAlpha = ft.life; ctx.fillStyle = ft.color;

```

```

1964     ctx.font = `bold ${14 + (1 - ft.life) * 6}px sans-serif`; ctx.textAlign = 'center';
1965     ctx.shadowColor = ft.color; ctx.shadowBlur = 8;
1966     ctx.fillText(ft.text, ft.x, ft.y); ctx.shadowBlur = 0;
1967 }
1968 ctx.globalAlpha = 1; ctx.textAlign = 'left';
1969
1970 // Field weapons – draw UNDER monsters so sprites don't block visibility
1971 for (const w of game.player.weapons) {
1972     if (w.type !== 'field') continue;
1973     const vs = w.vStyle || 'magnet';
1974     const px = Math.floor(game.player.x), py = Math.floor(game.player.y);
1975     const fr = Math.floor(w.fieldRadius);
1976     const phase = game.frame * 0.05;
1977
1978     // --- vStyle-based field drawing (sprite + fallback glow) ---
1979     const smap = WEAPON_SPRITE_MAP[vs];
1980     const sheet = smap ? SPRITE_SHEETS[smap.sheet] : null;
1981     const hasSprite = sheet && sheet.img;
1982     if (!hasSprite) {
1983         ctx.fillStyle = w.color + '22';
1984         ctx.beginPath(); ctx.arc(px, py, fr, 0, Math.PI * 2); ctx.fill();
1985         ctx.strokeStyle = w.color + 'aa'; ctx.lineWidth = 3;
1986         ctx.beginPath(); ctx.arc(px, py, fr, 0, Math.PI * 2); ctx.stroke();
1987     } else {
1988         // Semi-transparent so monsters underneath remain visible
1989         ctx.globalAlpha = 0.35;
1990         drawWeaponSprite(vs, px, py, fr * 2, fr * 2);
1991         ctx.globalAlpha = 1;
1992         // Subtle rim ring over the sprite
1993         ctx.strokeStyle = w.color + '66'; ctx.lineWidth = 1;
1994         ctx.beginPath(); ctx.arc(px, py, fr, 0, Math.PI * 2); ctx.stroke();
1995     }
1996
1997     // Universal perimeter sparks (for visual flair on all field weapons)
1998     for (let s = 0; s < (vs === 'starfield' ? 0 : vs === 'cosmic' ? 4 : 6); s++) {
1999         const angle = (s / 8) * Math.PI * 2 + phase;
2000         const sparkDist = fr + (Math.random() - 0.5) * 4;
2001         const cx = px + Math.cos(angle) * sparkDist;
2002         const cy = py + Math.sin(angle) * sparkDist;
2003         ctx.fillStyle = w.color;
2004         ctx.globalAlpha = 0.25 + Math.random() * 0.2;
2005         ctx.beginPath(); ctx.arc(cx, cy, 2, 0, Math.PI * 2); ctx.fill();
2006     }
2007     ctx.globalAlpha = 1;
2008 }
2009
2010 // Monsters
2011 for (const m of game.monsters) drawMonster(m);
2012
2013 // Orbital weapons
2014 for (const w of game.player.weapons) {
2015     if (w.type !== 'orbital') continue;
2016     const vs = w.vStyle || 'iron';
2017     for (let j = 0; j < w.count; j++) {
2018         const orbAngle = w.angle + (j / w.count) * Math.PI * 2;
2019         const ox = Math.floor(game.player.x + Math.cos(orbAngle) * w.orbitRadius);
2020         const oy = Math.floor(game.player.y + Math.sin(orbAngle) * w.orbitRadius);
2021         const sz = Math.floor(w.size);
2022         const orbGlow = Math.min(30, 10 + w.damage * 2);
2023
2024         // Orbit trail - pixel dots
2025         for (let t = 0; t < 5; t++) {
2026             const ta = orbAngle - t * 0.08;
2027             const tx = game.player.x + Math.cos(ta) * w.orbitRadius;
2028             const ty = game.player.y + Math.sin(ta) * w.orbitRadius;
2029             ctx.fillStyle = w.color + '44';
2030             ctx.fillRect(Math.floor(tx) - 1, Math.floor(ty) - 1, 3, 3);
2031         }
2032
2033         // Spark trail (damage-boosted orbitals)
2034         if (w.vfx.sparkTrail) {
2035             for (let s = 0; s < 3; s++) {
2036                 const sa = orbAngle - 0.3 + s * 0.3;
2037                 const sx = game.player.x + Math.cos(sa) * (w.orbitRadius - 4);
2038                 const sy = game.player.y + Math.sin(sa) * (w.orbitRadius - 4);
2039                 const sparkAlpha = 0.3 + Math.random() * 0.3;
2040                 ctx.fillStyle = 'ffffff';
2041                 ctx.globalAlpha = sparkAlpha;

```

```

2042         ctx.beginPath(); ctx.arc(sx + (Math.random()-0.5)*4, sy + (Math.random()-0.5)*4, 1
.5, 0, Math.PI*2); ctx.fill();
2043         ctx.globalAlpha = 1;
2044     }
2045 }
2046
2047 // Energy ring for epic orbitals
2048 if (w.vfx.energyRing) {
2049     ctx.strokeStyle = w.color + '55'; ctx.lineWidth = 2;
2050     ctx.beginPath(); ctx.arc(ox, oy, sz + 4, 0, Math.PI * 2); ctx.stroke();
2051 }
2052
2053 // --- vStyle-based orb drawing (sprite + fallback) ---
2054 const spriteDrawn = drawWeaponSprite(vs, ox, oy, sz * 2, sz * 2, orbAngle);
2055 if (!spriteDrawn) {
2056     ctx.shadowColor = w.color; ctx.shadowBlur = orbGlow;
2057     ctx.fillStyle = w.color;
2058     ctx.beginPath(); ctx.arc(ox, oy, sz, 0, Math.PI * 2); ctx.fill();
2059     ctx.fillStyle = 'ffffff';
2060     ctx.beginPath(); ctx.arc(ox, oy, sz * 0.4, 0, Math.PI * 2); ctx.fill();
2061     ctx.shadowBlur = 0;
2062 }
2063 }
2064 }
2065
2066 // Projectiles
2067 for (const pr of game.projectiles) {
2068     const hasVfx = pr.vfx || {};
2069     const vs = pr.vStyle || 'fire';
2070     // Trail
2071     for (const t of pr.trail) { ctx.globalAlpha = t.life * 0.35; ctx.fillStyle = pr.color; c
tx.beginPath(); ctx.arc(t.x, t.y, pr.radius * t.life * 0.8, 0, Math.PI * 2); ctx.fill(); }
2072     ctx.globalAlpha = 1;
2073
2074     // Heat wave distortion ring (cooldown boost)
2075     if (hasVfx.heatWave) {
2076         ctx.strokeStyle = pr.color + '44'; ctx.lineWidth = 1.5;
2077         ctx.beginPath(); ctx.arc(pr.x, pr.y, pr.radius + 5, 0, Math.PI * 2); ctx.stroke();
2078     }
2079
2080     // Energy ring (damage boost)
2081     if (hasVfx.energyRing) {
2082         const erLife = ((Date.now() / 200) % 1);
2083         ctx.strokeStyle = pr.color + '88'; ctx.lineWidth = 2;
2084         ctx.beginPath(); ctx.arc(pr.x, pr.y, pr.radius + erLife * 6, 0, Math.PI * 2); ctx.stro
ke();
2085     }
2086
2087     const px = Math.floor(pr.x), py = Math.floor(pr.y);
2088     const r = Math.floor(pr.radius);
2089     const s = Math.floor(pr.radius * 1.6);
2090
2091     // --- vStyle-based visual (sprite + fallback) ---
2092     const pAngle = Math.atan2(pr.vy, pr.vx);
2093     const spriteSize = Math.max(s, 16);
2094     const spriteDrawn = drawWeaponSprite(vs, px, py, spriteSize, spriteSize, pAngle);
2095     if (!spriteDrawn) {
2096         ctx.fillStyle = pr.color;
2097         ctx.fillRect(px - s/2, py - s/2, s, s);
2098         ctx.fillStyle = 'ffffff';
2099         const coreSize = Math.floor(r * 0.6);
2100         ctx.fillRect(px - coreSize/2, py - coreSize/2, coreSize, coreSize);
2101     }
2102
2103     // Piercing glow - outer ring
2104     if (hasVfx.piercingGlow) {
2105         ctx.strokeStyle = 'ffffff'; ctx.lineWidth = 2;
2106         ctx.strokeRect(px - s/2 - 2, py - s/2 - 2, s + 4, s + 4);
2107     }
2108 }
2109
2110 // Player
2111 if (game.state !== 'dying' || game.deathAnim < 10) {
2112     // Dash afterimages
2113     for (const ai of game.player.afterimages) {
2114         ctx.globalAlpha = ai.life * 0.35;
2115         const ps = SPRITE_SHEETS.player;
2116         if (ps && ps.img) {

```

```

2117         const afr = Math.floor(game.frame / 6) % ps.cols;
2118         ctx.save();
2119         ctx.imageSmoothingEnabled = false;
2120         ctx.translate(Math.floor(ai.x), Math.floor(ai.y));
2121         ctx.scale(game.player._facingRight !== false ? 1 : -1, 1);
2122         ctx.drawImage(ps.img, afr * ps.cellW, 0, ps.cellW, ps.cellH, -ps.cellW / 2, -ps.cell
H / 2, ps.cellW, ps.cellH);
2123         ctx.restore();
2124     } else {
2125         ctx.fillStyle = '#ffd700';
2126         ctx.fillRect(Math.floor(ai.x) - 11, Math.floor(ai.y) - 14, 22, 28);
2127     }
2128 }
2129 ctx.globalAlpha = 1;
2130 drawPlayer();
2131 }
2132 // Level up flash
2133 if (game.flashTimer > 0) {
2134     const p = game.player, r = (20 - game.flashTimer) * 12;
2135     ctx.strokeStyle = 'rgba(255, 215, 0, ' + (game.flashTimer / 20).toFixed(2) + ')'; ctx.li
newWidth = 4;
2136     ctx.strokeRect(p.x - r, p.y - r, r * 2, r * 2);
2137 }
2138
2139 ctx.restore();
2140
2141 // Joystick
2142 if (joy.active) drawJoystick();
2143
2144 // Danger vignette
2145 drawDangerVignette();
2146
2147 // Minimap
2148 drawMinimap();
2149 }
2150
2151 function drawMap() {
2152     const cam = game.camera;
2153     // Background image tiling
2154     if (bgImg && bgImg.ready) {
2155         const ts = bgImg.width;
2156         const gx0 = Math.floor(cam.x / ts) * ts, gy0 = Math.floor(cam.y / ts) * ts;
2157         for (let gy = gy0; gy < cam.y + H + ts; gy += ts)
2158             for (let gx = gx0; gx < cam.x + W + ts; gx += ts)
2159                 if (gx >= -ts && gx < CFG.MAP_W + ts && gy >= -ts && gy < CFG.MAP_H + ts)
2160                     ctx.drawImage(bgImg, gx, gy);
2161     } else {
2162         const ts = skyTex.width;
2163         const gx0 = Math.floor(cam.x / ts) * ts, gy0 = Math.floor(cam.y / ts) * ts;
2164         for (let gy = gy0; gy < cam.y + H + ts; gy += ts)
2165             for (let gx = gx0; gx < cam.x + W + ts; gx += ts)
2166                 if (gx >= -ts && gx < CFG.MAP_W + ts && gy >= -ts && gy < CFG.MAP_H + ts)
2167                     ctx.drawImage(skyTex, gx, gy);
2168     }
2169
2170 // Obstacles – bricks & pipes
2171 for (const o of game.obstacles) {
2172     if (o.x + o.w < cam.x - 20 || o.x > cam.x + W + 20 ||
2173         o.y + o.h < cam.y - 20 || o.y > cam.y + H + 20) continue;
2174     if (o.type === 'brick') {
2175         // Brick block
2176         ctx.fillStyle = '#c84c09'; ctx.fillRect(o.x, o.y, o.w, o.h);
2177         // Brick pattern
2178         ctx.strokeStyle = '#6b3000'; ctx.lineWidth = 1;
2179         ctx.strokeRect(o.x, o.y, o.w, o.h);
2180         const bw = o.w / 3;
2181         for (let r = 0; r < Math.floor(o.h / 8); r++) {
2182             const ry = o.y + r * 8;
2183             for (let c = 0; c < 3; c++) {
2184                 const bx = o.x + c * bw + (r % 2) * (bw / 2);
2185                 ctx.strokeStyle = '#8b4513'; ctx.lineWidth = 0.5;
2186                 ctx.strokeRect(bx, ry, bw, 7);
2187             }
2188         }
2189         // Top edge highlight
2190         ctx.fillStyle = '#e87830';
2191         ctx.fillRect(o.x + 1, o.y + 1, o.w - 2, 3);
2192     } else {

```

```

2193     // Pipe
2194     ctx.fillStyle = '#38a838'; ctx.fillRect(o.x, o.y, o.w, o.h);
2195     // Pipe rim
2196     ctx.fillStyle = '#50c850'; ctx.fillRect(o.x - 4, o.y, o.w + 8, 8);
2197     ctx.strokeStyle = '#207820'; ctx.lineWidth = 2;
2198     ctx.strokeRect(o.x - 4, o.y, o.w + 8, o.h);
2199     // Pipe shading
2200     ctx.fillStyle = 'rgba(255,255,255,0.12)';
2201     ctx.fillRect(o.x + 3, o.y + 8, 8, o.h - 8);
2202     ctx.fillStyle = 'rgba(0,0,0,0.15)';
2203     ctx.fillRect(o.x + o.w - 10, o.y + 8, 7, o.h - 8);
2204 }
2205 }
2206 ctx.strokeStyle = '#8b4513'; ctx.lineWidth = 8;
2207 ctx.strokeRect(0, 0, CFG.MAP_W, CFG.MAP_H);
2208 ctx.strokeStyle = '#a0522d'; ctx.lineWidth = 4;
2209 ctx.strokeRect(4, 4, CFG.MAP_W - 8, CFG.MAP_H - 8);
2210 }
2211
2212 function drawPlayer() {
2213     const p = game.player, r = CFG.PLAYER_RADIUS;
2214     const moveX = (keys.d ? 1 : 0) - (keys.a ? 1 : 0) + joy.dx;
2215     const moveY = (keys.s ? 1 : 0) - (keys.w ? 1 : 0) + joy.dy;
2216     const moving = Math.abs(moveX) > 0.1 || Math.abs(moveY) > 0.1;
2217     if (moveX > 0.1) p._facingRight = true;
2218     else if (moveX < -0.1) p._facingRight = false;
2219     const facingRight = p._facingRight !== false;
2220     const bob = moving ? Math.sin(game.frame * 0.15) * 2 : 0;
2221     const cpx = Math.floor(p.x), cpy = Math.floor(p.y + bob);
2222
2223     const sheet = SPRITE_SHEETS.player;
2224     const cw = sheet ? sheet.cellW : 64;
2225     const ch = sheet ? sheet.cellH : 64;
2226     const totalFrames = sheet ? sheet.cols : 1;
2227     const animFrame = totalFrames > 1 ? (Math.floor(game.frame / 6) % totalFrames) : 0;
2228     const invert = game.player.invulnFrames > 0 && Math.floor(game.player.invulnFrames / 3) %
2229 2 === 0;
2230
2231     // Shadow
2232     ctx.fillStyle = 'rgba(0,0,0,0.25)';
2233     ctx.fillRect(cpx - 12, cpy + 30, 24, 4);
2234
2235     // Emperor player sprite (flipped by facing direction)
2236     if (sheet && sheet.img) {
2237         ctx.save();
2238         ctx.imageSmoothingEnabled = false;
2239         ctx.translate(cpx, cpy);
2240         ctx.scale(facingRight ? 1 : -1, 1);
2241         if (invert) ctx.filter = 'brightness(0) invert(1)';
2242         ctx.drawImage(sheet.img, animFrame * cw, 0, cw, ch, -cw / 2, -ch / 2, cw, ch);
2243         ctx.restore();
2244     } else {
2245         // Fallback circle while sprite loads
2246         ctx.fillStyle = invert ? 'ffffff' : '#e82000';
2247         ctx.beginPath(); ctx.arc(cpx, cpy, r, 0, Math.PI * 2); ctx.fill();
2248     }
2249
2250     // Dash / invuln glow
2251     if (game.player.invulnFrames > 0 && !invert) {
2252         ctx.strokeStyle = '#ffd700'; ctx.lineWidth = 3;
2253         ctx.strokeRect(cpx - 34, cpy - 34, 68, 68);
2254     }
2255 }
2256
2257 function drawMonster(m) {
2258     const scale = m.spawnAnim > 0 ? 1 - m.spawnAnim : 1, r = m.radius * scale;
2259     const sx = Math.floor(m.x), sy = Math.floor(m.y);
2260     const flash = m.flash > 0;
2261     const hitTint = flash;
2262
2263     // Shadow
2264     ctx.fillStyle = 'rgba(0,0,0,0.25)';
2265     ctx.fillRect(sx - r * 0.5, sy + r * 0.6, r, 4);
2266
2267     // Elite aura
2268     if (m.isElite) {
2269         const pulse = 0.6 + Math.sin(game.frame * 0.1) * 0.2;
2270         ctx.strokeStyle = 'rgba(255, 215, 0, ' + pulse.toFixed(2) + ')'; ctx.lineWidth = 3;

```



```

2270     ctx.strokeRect(sx - r - 4, sy - r - 4, (r + 4) * 2, (r + 4) * 2);
2271     ctx.fillStyle = '#ffd700';
2272     ctx.fillRect(sx - 4, sy - r - 12, 8, 6);
2273     ctx.fillRect(sx - 6, sy - r - 10, 12, 2);
2274     for (let i = 0; i < 3; i++) ctx.fillRect(sx - 3 + i * 3, sy - r - 16, 2, 5);
2275 }
2276
2277 const eyeAng = Math.atan2(game.player.y - sy, game.player.x - sx);
2278 const eDX = Math.round(Math.cos(eyeAng)), eDY = Math.round(Math.sin(eyeAng));
2279
2280 switch (m.shape) {
2281     case 'blob': { // Goomba - brown mushroom
2282         ctx.fillStyle = hitTint ? '#ffffff' : '#b87830';
2283         ctx.fillRect(sx - r, sy - r * 0.3, r * 2, r * 1.1);
2284         ctx.fillStyle = hitTint ? '#ffffff' : '#8b5518';
2285         ctx.beginPath(); ctx.arc(sx, sy - r * 0.3, r * 0.85, Math.PI, 0); ctx.fill();
2286         ctx.fillStyle = hitTint ? '#ffffff' : '#6b3300';
2287         ctx.fillRect(sx - r + 2, sy + r * 0.6, r * 0.6, r * 0.3);
2288         ctx.fillRect(sx + r * 0.2, sy + r * 0.6, r * 0.6, r * 0.3);
2289         ctx.fillStyle = '#ffffff';
2290         ctx.fillRect(sx - r * 0.5, sy - r * 0.3, 4, 5);
2291         ctx.fillRect(sx + r * 0.2, sy - r * 0.3, 4, 5);
2292         ctx.fillStyle = '#111';
2293         ctx.fillRect(sx - r * 0.3, sy - r * 0.1, 2, 3);
2294         ctx.fillRect(sx + r * 0.4, sy - r * 0.1, 2, 3);
2295         break;
2296     }
2297     case 'diamond': { // Red Koopa-like
2298         ctx.fillStyle = hitTint ? '#ffffff' : '#58b828';
2299         ctx.beginPath(); ctx.ellipse(sx, sy, r * 0.8, r * 0.55, 0, 0, Math.PI * 2); ctx.fill();
2300
2301         ctx.fillStyle = hitTint ? '#ffffff' : '#3a9018';
2302         ctx.fillRect(sx - r * 0.4, sy - r * 0.25, r * 0.8, r * 0.5);
2303         ctx.fillStyle = hitTint ? '#ffffff' : '#f0d800';
2304         ctx.fillRect(sx + r * 0.3, sy - r * 0.5, r * 0.5, r * 0.5);
2305         ctx.fillRect(sx - r * 0.6, sy + r * 0.3, r * 0.4, r * 0.3);
2306         ctx.fillRect(sx + r * 0.2, sy + r * 0.3, r * 0.4, r * 0.3);
2307         ctx.fillStyle = '#ffffff';
2308         ctx.fillRect(sx + r * 0.5, sy - r * 0.4, 4, 4);
2309         ctx.fillRect(sx + r * 0.7, sy - r * 0.4, 4, 4);
2310         ctx.fillStyle = '#111';
2311         ctx.fillRect(sx + r * 0.6, sy - r * 0.3, 2, 2);
2312         break;
2313     }
2314     case 'square': { // Green Shy Guy
2315         ctx.fillStyle = hitTint ? '#ffffff' : '#58b828';
2316         ctx.fillRect(sx - r * 0.6, sy - r * 0.1, r * 1.2, r * 0.7);
2317         ctx.fillStyle = hitTint ? '#ffffff' : '#f0e8d0';
2318         ctx.beginPath(); ctx.arc(sx, sy - r * 0.25, r * 0.5, 0, Math.PI * 2); ctx.fill();
2319         ctx.fillStyle = '#111';
2320         ctx.fillRect(sx - r * 0.25, sy - r * 0.35, 2, 2);
2321         ctx.fillRect(sx + r * 0.15, sy - r * 0.35, 2, 2);
2322         ctx.fillRect(sx - 2, sy - r * 0.1, 4, 1);
2323         ctx.fillStyle = hitTint ? '#ffffff' : '#6b3300';
2324         ctx.fillRect(sx - r * 0.45, sy + r * 0.4, r * 0.4, r * 0.25);
2325         ctx.fillRect(sx + r * 0.05, sy + r * 0.4, r * 0.4, r * 0.25);
2326         break;
2327     }
2328     case 'circle': { // Boo - white ghost
2329         ctx.fillStyle = hitTint ? '#ffffff' : 'rgba(240,240,255,0.8)';
2330         ctx.beginPath(); ctx.arc(sx, sy - r * 0.1, r * 0.7, Math.PI, 0);
2331         for (let i = 0; i <= 4; i++) {
2332             const wx = sx - r * 0.7 + (i / 4) * r * 1.4;
2333             const wy = sy + r * 0.5 + (i % 2 === 0 ? r * 0.2 : 0);
2334             if (i === 0) ctx.lineTo(wx, wy); else ctx.lineTo(wx, wy);
2335         }
2336         ctx.closePath(); ctx.fill();
2337         ctx.fillStyle = '#111';
2338         ctx.fillRect(sx - r * 0.35, sy - r * 0.25, r * 0.3, r * 0.35);
2339         ctx.fillRect(sx + r * 0.05, sy - r * 0.25, r * 0.3, r * 0.35);
2340         ctx.fillRect(sx - r * 0.15, sy + r * 0.05, r * 0.3, r * 0.2);
2341         break;
2342     }
2343     case 'triangle': { // Bowser-like
2344         ctx.fillStyle = hitTint ? '#ffffff' : '#e89820';
2345         ctx.fillRect(sx - r * 0.7, sy - r * 0.2, r * 1.4, r * 0.8);
2346         ctx.fillStyle = hitTint ? '#ffffff' : '#58a828';
2347         ctx.beginPath(); ctx.ellipse(sx, sy, r * 0.65, r * 0.45, 0, Math.PI, 0); ctx.fill();

```

```

2347     ctx.fillStyle = hitTint ? '#ffffff' : '#ffffff';
2348     for (let i = 0; i < 3; i++) {
2349         ctx.fillRect(sx - r * 0.4 + i * r * 0.4, sy - r * 0.8, r * 0.15, r * 0.35);
2350     }
2351     ctx.fillStyle = hitTint ? '#ffffff' : '#58a828';
2352     ctx.fillRect(sx - r * 0.35, sy - r * 0.7, r * 0.7, r * 0.4);
2353     ctx.fillStyle = hitTint ? '#ffffff' : '#e0e0e0';
2354     ctx.fillRect(sx - r * 0.4, sy - r * 1, 3, r * 0.3);
2355     ctx.fillRect(sx + r * 0.3, sy - r * 1, 3, r * 0.3);
2356     ctx.fillStyle = '#ff0000';
2357     ctx.fillRect(sx - r * 0.25, sy - r * 0.6, 4, 4);
2358     ctx.fillRect(sx + r * 0.1, sy - r * 0.6, 4, 4);
2359     ctx.fillStyle = hitTint ? '#ffffff' : '#e89820';
2360     ctx.fillRect(sx - r * 0.85, sy - r * 0.2, r * 0.3, r * 0.5);
2361     ctx.fillRect(sx + r * 0.55, sy - r * 0.2, r * 0.3, r * 0.5);
2362     break;
2363 }
2364 case 'star': { // King Boo
2365     ctx.fillStyle = hitTint ? '#ffffff' : '#ffd700';
2366     ctx.fillRect(sx - r * 0.35, sy - r * 1.1, r * 0.7, r * 0.2);
2367     for (let i = 0; i < 3; i++) ctx.fillRect(sx - r * 0.2 + i * r * 0.2, sy - r * 1.3, r *
0.1, r * 0.25);
2368     ctx.fillStyle = hitTint ? '#ffffff' : 'rgba(120, 80, 180, 0.85)';
2369     ctx.beginPath(); ctx.arc(sx, sy - r * 0.1, r * 0.75, Math.PI, 0);
2370     for (let i = 0; i <= 5; i++) {
2371         const wx = sx - r * 0.75 + (i / 5) * r * 1.5;
2372         const wy = sy + r * 0.5 + (i % 2 === 0 ? r * 0.25 : 0);
2373         if (i === 0) ctx.lineTo(wx, wy); else ctx.lineTo(wx, wy);
2374     }
2375     ctx.closePath(); ctx.fill();
2376     ctx.fillStyle = '#ffffff'; ctx.shadowColor = '#000'; ctx.shadowBlur = 4;
2377     ctx.fillRect(sx - r * 0.4, sy - r * 0.35, r * 0.3, r * 0.35);
2378     ctx.fillRect(sx + r * 0.1, sy - r * 0.35, r * 0.3, r * 0.35);
2379     ctx.fillStyle = '#111'; ctx.shadowBlur = 0;
2380     ctx.fillRect(sx - r * 0.25 + eDX, sy - r * 0.25 + eDY, 3, 3);
2381     ctx.fillRect(sx + r * 0.25 + eDX, sy - r * 0.25 + eDY, 3, 3);
2382     const barW = r * 1.6;
2383     ctx.fillStyle = 'rgba(0,0,0,0.6)'; ctx.fillRect(sx - barW / 2, sy - r - 16, barW, 5);
2384     ctx.fillStyle = '#ff00ff'; ctx.fillRect(sx - barW / 2, sy - r - 16, barW * (m.hp / m.m
axHp), 5);
2385     break;
2386 }
2387 }
2388 // Health bar – show once monster takes damage
2389 if (m.hp < m.maxHp) {
2390     const barWx = Math.max(8, r * 1.5);
2391     const barHx = 3;
2392     const barY = m.isElite ? sy - r - 22 : sy - r - 10;
2393     const hpRatio = m.hp / m.maxHp;
2394     ctx.fillStyle = 'rgba(0,0,0,0.5)';
2395     ctx.fillRect(sx - barWx / 2, barY, barWx, barHx);
2396     const hpColor = hpRatio > 0.6 ? '#44ff44' : hpRatio > 0.3 ? '#ffaa00' : '#ff4444';
2397     ctx.fillStyle = hpColor;
2398     ctx.fillRect(sx - barWx / 2, barY, barWx * hpRatio, barHx);
2399 }
2400 }
2401 function drawJoystick() {
2402     const x = joy.cx, y = joy.cy;
2403     ctx.strokeStyle = 'rgba(139, 69, 19, 0.35)'; ctx.lineWidth = 2; ctx.beginPath(); ctx.arc(x
, y, 60, 0, Math.PI * 2); ctx.stroke();
2404     ctx.fillStyle = 'rgba(139, 69, 19, 0.3)'; ctx.beginPath(); ctx.arc(x + joy.dx, y + joy.dy,
25, 0, Math.PI * 2); ctx.fill();
2405     ctx.strokeStyle = 'rgba(139, 69, 19, 0.6)'; ctx.lineWidth = 2; ctx.beginPath(); ctx.arc(x
+ joy.dx, y + joy.dy, 25, 0, Math.PI * 2); ctx.stroke();
2406 }
2407
2408 function drawDangerVignette() {
2409     if (game.state !== 'playing') return;
2410     let minDist = Infinity;
2411     for (const m of game.monsters) { const d = Math.hypot(m.x - game.player.x, m.y - game.play
er.y); if (d < minDist) minDist = d; }
2412     const dangerRange = 120;
2413     if (minDist < dangerRange) { const intensity = 1 - minDist / dangerRange; const grad = ctx
.createRadialGradient(W / 2, H / 2, Math.min(W, H) * 0.3, W / 2, H / 2, Math.max(W, H) * 0.7
); grad.addColorStop(0, 'rgba(255, 0, 0, 0)'); grad.addColorStop(1, `rgba(255, 0, 0, ${inten
sity * 0.3})`); ctx.fillStyle = grad; ctx.fillRect(0, 0, W, H); }
2414 }
2415

```

```

2416 function drawMinimap() {
2417     const mw = 120, mh = 120;
2418     miniCtx.clearRect(0, 0, mw, mh); miniCtx.fillStyle = 'rgba(92, 148, 252, 0.7)'; miniCtx.fi
    llRect(0, 0, mw, mh);
2419     const scaleX = mw / CFG.MAP_W, scaleY = mh / CFG.MAP_H;
2420     miniCtx.strokeStyle = 'rgba(139, 69, 19, 0.5)'; miniCtx.lineWidth = 1; miniCtx.strokeRect(
    1, 1, mw - 2, mh - 2);
2421     for (const m of game.monsters) { const mx = m.x * scaleX, my = m.y * scaleY; miniCtx.fillS
    tyle = m.isBoss ? '#8850b8' : (m.isElite ? '#ffd700' : m.color); const size = m.isBoss ? 3 :
    (m.isElite ? 2.5 : 1.5); miniCtx.fillRect(mx - size / 2, my - size / 2, size, size); }
2422     for (const o of game.obstacles) {
2423         const ox = o.x * scaleX, oy = o.y * scaleY, ow = Math.max(1, o.w * scaleX), oh = Math.ma
    x(1, o.h * scaleY);
2424         miniCtx.fillStyle = o.type === 'brick' ? 'rgba(200,100,30,0.6)' : 'rgba(56,168,56,0.6)';
2425         miniCtx.fillRect(ox, oy, ow, oh);
2426     }
2427     const px = game.player.x * scaleX, py = game.player.y * scaleY;
2428     miniCtx.fillStyle = '#e82000'; miniCtx.fillRect(px - 2, py - 2, 5, 5);
2429     miniCtx.strokeStyle = 'rgba(255, 215, 0, 0.5)'; miniCtx.lineWidth = 1; miniCtx.beginPath()
    ; miniCtx.arc(px, py, 8, 0, Math.PI * 2); miniCtx.stroke();
2430     const cx = game.camera.x * scaleX, cy = game.camera.y * scaleY, cw = W * scaleX, ch = H *
    scaleY;
2431     miniCtx.strokeStyle = 'rgba(255, 255, 255, 0.15)'; miniCtx.lineWidth = 1; miniCtx.strokeRe
    ct(cx, cy, cw, ch);
2432 }
2433
2434 // ---- Game Loop ----
2435 function loop() { update(); render(); requestAnimationFrame(loop); }
2436
2437 function startGame() {
2438     initAudio();
2439     initGame();
2440     document.getElementById('menu').classList.add('hidden');
2441     document.getElementById('gameover').classList.remove('show');
2442     document.getElementById('joyHint').style.display = 'none';
2443     dashBtnEl.style.display = '';
2444     hideUpgradeModal();
2445     startBgMusic();
2446 }
2447
2448 document.getElementById('btnStart').addEventListener('click', startGame);
2449 document.getElementById('btnRetry').addEventListener('click', startGame);
2450
2451 // ---- Device detection & dash button setup ----
2452 const isTouchDevice = ('ontouchstart' in window) || (navigator.maxTouchPoints > 0);
2453 const dashBtnEl = document.getElementById('dashBtn');
2454 const hudDashHint = document.getElementById('hudDash');
2455
2456 // On touch devices: hide "SPACE" text hint, show big dash button
2457 // On desktop: keep SPACE hint, also show dash button (clickable as alternative)
2458 if (isTouchDevice) {
2459     hudDashHint.style.display = 'none';
2460 } else {
2461     // Desktop: keep both, dash button also responds to click
2462     hudDashHint.style.display = '';
2463 }
2464
2465 // Mouse click on dash button (desktop alternative)
2466 dashBtnEl.addEventListener('click', () => {
2467     if (game && game.state === 'playing' && !game.upgradeActive) tryDash();
2468 });
2469 // Dash button touch handling (independent of canvas, supports multi-touch)
2470 dashBtnEl.addEventListener('touchstart', e => {
2471     e.preventDefault();
2472     e.stopPropagation();
2473     if (game && game.state === 'playing' && !game.upgradeActive) {
2474         tryDash();
2475         dashBtnEl.classList.add('pressed');
2476     }
2477 }, { passive: false });
2478 dashBtnEl.addEventListener('touchend', e => {
2479     e.preventDefault();
2480     e.stopPropagation();
2481     dashBtnEl.classList.remove('pressed');
2482 }, { passive: false });
2483 dashBtnEl.addEventListener('touchcancel', () => {
2484     dashBtnEl.classList.remove('pressed');
2485 }, { passive: false });

```

```
2486
2487  showMenuBest();
2488  initGame(); game.state = 'menu'; loop();
2489  </script>
2490  </body>
2491  </html>
2492
```